Undecidability

Maximilian Siemers

Version: 14, 2026-04-13

Course mechanics

Instructor:	Max(imilian Siemers)
E-Mail:	msiemers@cmu.edu
Meetings:	Tuesday/Thursday 11-12.20, Wean 5328
Lecture notes:	Link on Canvas. Will be updated frequently! (work in progress)
Office hours:	Tuesday 3pm, Baker Hall 155E
Assignments:	(Bi-)Weekly, posted on Canvas. Assigned Thursdays, due Thursdays **before class**
Assessment:	Assignments: 35%, Final exam: 15%
Final exam time:	Friday, May 1st, 8.30-11.30am
Final exam mode:	Individual oral exam

Sources

I will make lecture notes available on Canvas. These are work in progress and will be updated frequently.

The lectures notes are based on the following sources, which I will reference using the indicated codes:

- [BBJ] Boolos, G. S., Burgess, J. P., and Jeffrey, R. (2007): *Computability and Logic* (5th ed.). Cambrige University Press
- [IC] Open Logic Project (2025): *Incompleteness and Computability*
  Available here: https://ic.openlogicproject.org/
- [J] Avigad, Jeremy (2007): *Computability and Incompleteness. Lecture notes*
  Available here: https://www.andrew.cmu.edu/user/avigad/Teaching/candi_notes.pdf

Contents

A toy example

Consider the following little puzzle from Douglas Hofstadter's *Gödel, Escher, Bach* (1979): There are three symbols M, I, and U and four rules:

1. $xI \to xIU$

   (Add a U to the end of any string ending in I. Example: MI to MIU)

2. $Mx \to Mxx$

   (Double the string after an initial M. Example: MIU to $MIUIU$)

3. $xIIIy \to xUy$

   (Replace any III with a U. Example: $MUIIIU$ to $MUUU$)

4. $xUUy \to xy$

   (Remove any UU. Example: $MUUU$ to MU)

Question: Using these rules, can you transform the string MI to the string MU?

1 Introduction

In the first half of this course, you proved Gödel's Incompleteness Theorems, two groundbreaking limitational results that concern certain formal systems. These results say something about what can *not* be proved in formal systems that contain basic arithmetic (Robinson's Q including zero, successor, addition, multiplication). You saw that basic arithmetic is expressive enough to "talk about itself" in a certain sense: for example, the notion of "proof", "provability", and "consistency" can be *represented* by formulas in the language of arithmetic. This gives rise to certain paradoxa essentially due to self-reference. For example, it is possible to formulate *in the language of arithmetic* so-called *Gödel sentences*, which essentially say: "This sentence is not provable". And so we get:

Theorem 1.1 (Gödel's First Incompleteness Theorem)**.** *There is no consistent, complete, axiomatizable extension of Q.*

Theorem 1.2 (Gödel's Second Incompleteness Theorem)**.** *No consistent axiomatizable extension of Q derives its own consistency sentence.*

These results state limitations of what can be *proved* in logical systems rich enough to talk about arithmetic. In the second half of the course, we shift focus and investigate another aspect of logical systems: namely, to what extent we can devise algorithms that can *compute* certain properties of these systems or answer certain questions about them.

The central result we will prove is the *unsolvability of the decision problem for first-order logic*. The decision problem for first-order logic in its modern form was posed by German mathematician **David Hilbert** in 1928, but it traces back farther in history. Already in the 17th century, German philosopher and polymath **Gottfried Leibniz** dreamt of a machine that can automatically and through mere symbol manipulation decide for any given mathematical statement whether it was true or false. In the 1920s and 1930s, the decision problem was one of the most important open questions in the foundations of math, and occupied the minds of the leading mathematicians of the time – in particular those around David Hilbert in Göttingen. Since that time, it is – even in English – often called by its German name: **Entscheidungsproblem**.

> **Entscheidungsproblem (Decision problem for first-order logic).**
> Is there an algorithm that, given any sentence φ in the language of first-order logic, determines whether φ valid – i.e., true in all models?

Recall the **soundness** and **completeness** results for first-order logic: A proof system for first-order logic (e.g., natural deduction) is **sound** if $\vdash \varphi \implies \vDash \varphi$ (if φ can be proved, then it is valid); it is **complete** if $\vDash \varphi \implies \vdash \varphi$ (if φ is valid, then it can be proved). Soundness and completeness together mean that $\vdash \varphi \iff \vDash \varphi$ – i.e., provability and validity coincide.

Thus, by soundness and completeness, the decision problem for first-order logic can be equivalently stated as follows:

> **Entscheidungsproblem (Decision problem for first-order logic).**
> Is there an algorithm that, given any sentence φ in the language of first-order logic, determines whether φ is provable?

Think for a moment what it would mean if the answer to the Entscheidungsproblem was *yes*. Suppose there was a machine M that, given any first-order sentene φ as input, is guaranteed to correctly determine after a finite amount of time whether or not φ is provable (valid) – say, by outputting 1 if the answer is "yes", and 0 if the answer is "no". A moment of thought shows that, although there are infinitely many sentences in the language of first-order logic, we can *enumerate* them. That is, we can devise a list $\varphi_1, \varphi_2, \ldots$ that contains all sentences of first-order logic[1]. Given the machine M and such an enumeration of sentences, we can start with φ_1, "feed" it to M, and wait for its output - by assumption, M after a finite amount of time will output either 1 (if $\vdash \varphi$) or 0 (if $\nvdash \varphi$). Once that is done, we continue with φ_2, ad infinitum.

[1] This depends on the specific *language* or *signature* we're working with, and such an enumeration of all sentences of first-order logic is possibly only under (very reasonable) assumptions about the underlying language: namely, that the language contains at most a countable infinity of non-logical symbols (function symbols, predicate symbols, and constants).

Of course, this process will take an infinite amount of time, but we will reach each formula after a *finite* amount of time and obtain a decision whether or not that formula is provable/valid. So we would have a theorem-prover that would (given infinite time) eventually list all the provable sentences of first-order logic. It is easy to show that this extends to all finitely axiomatizable theories of first-order logic.

In the mid-1930s, Alonzo Church and Alan Turing independently proved that the **Entscheidungsproblem is unsolvable**, or, as one says, that **first-order logic is undecidable**: There is **no algorithm** that can decide for an arbitrary first-order sentence φ whether or not φ is valid (provable). This, together with Gödel's Incompleteness Theorems that came about five years earlier, was a **huge** blow to many of the leading mathematicians of the time, for it means that in a certain sense logic (and, by extension, mathematics) eludes "automatic" treatment through algorithms. And this result is what we're going to prove in the second half of this course!

Note that in my above formulation of the unsolvability of the Entscheidungsproblem, I said that there is *no algorithm* that can (...). I have not made clear what I mean by "algorithm", but the result depends on that notion. For the negative result of the unsolvability of the Entscheidungsproblem says that something can *not* be computed. To make sense of the claim that something is or is not computable, we need to know what it means to be computable. Indeed, in Turing's 1936 paper *On computable numbers, with an application to the Entscheidungsproblem*, he introduces *Turing machines*, which are now considered *the* universal model of computation, in the following sense: **If** something can be computed at all, then it can be computed by a Turing machine. (In fact, Turing machines even provided the theoretical foundations for modern-day all-purpose computers, the first instantiations of which were constructed a little later.)

En route to proving undecidability of first-order logic, we will prove the undecidability of the famous *Halting problem* for Turing machines, and be concerned with the more general question: *What is computation?* We will introduce Turing machines and prove undecidability using them, but we will also see various other models of computation and prove their equivalence. The equivalence of a large number of *very* different models of computation gives rise to the **Church-Turing Thesis**, which is a thesis about what, precisely, computation *is*.

Thus, we will also deal with the fundamental issue of the *nature of computation*, and see multiple other exapmles of *(computably) unsolvable problems*. Given that you are at CMU, you have probably heard of the Church-Turing Thesis before, and many of you should already be convinced by the relevance of this issue. But just in case, here are a few further motivations:

- Modern *complexity theory*, which, for example, provides the theoretical groundworks for many approaches in *crpytography*, in a certain sense extends *computability theory*: Whereas computability deals with questions of what, in principle, is or is not computable, complexity theory adds constraints concerning

time and memory use in computation. The methods of complexity theory basically extend those of computability theory. Any of you who have taken a course in complexity theory will probably have dealt a lot with proofs involving Turing machines.

- Limits to *automated theorem proving*: Undecidability of first-order logic says that there can be no automated procedure that decides whether a given sentence of first-order logic is provable. Thus, there can also be no fully automated procedure that decides whether a given mathematical statement (formalized in first-order logic) is or is not a theorem.

- Limits to *software verification*: Undecidability of first-order logic has direct limitational consequences for the possibility to decise automated procedures that answer questions about computer programs such as:

  - Will this program ever crash?
  - Will this program ever reach a bad state?
  - Are these two programs equivalent?

  If time allows, we will prove *Rice's Theorem*, which states roughly that no non-trivial semantic property of programs is decidable.

- Scope of Gödel's incompleteness theorems: As stated above, Gödel's incompleteness theorems state something about *axiomatizable* theories. A theory is *axiomatizable* if there exists an algorithm that, for a given sentence, decides after a finite amount of time whether or not is is an axiom of the theory. Thus, the scope of Gödel's theorems directly depends on the question: *What is computation?*

One last remark before we really get started: Remember that, in proving incompleteness, we used the fact that basic arithmetic is expressive enough to express meta-logical properties of theories, derivations, provability, etc. and thus about itself. Something conceptually very simular is going on in the proof of the undecidability of the Halting problem and the undecidability of first-order logic: namely, first-order logic is expressive enough to "talk about" computation, as we will make precise.

2 Effective computability

Since the central results we will be concerned with in this part of the course are about problems that *can* or can *not* be solved by algorithms, we should first state what we mean by an *algorithm*. Intuitively, an algorithm (or a computational procedure) consists of a *recipe*, or a set of *instructions* that one (a person *or* a machine) can follow to obtain an answer to a question, or an output given an input.

So we are looking for a characterization of certain ("computable") maps from questions to answers, or from inputs to outputs. Thus it makes sense to think of algorithms as certain kinds of functions, and the question becomes: *what properties must a function have to be called "computable"*? For simplicity, let's just think about functions from natural numbers to natural numbers[2].

Historically, functions which are intuitively computable were called "effectively computable" or just "effective" functions. Think of this as meaning the same as "intuitively computable" functions. So – what is a "computable" function? Here is a nice description from [BBJ, p. 23]:

> A function f from positive integers to positive integers is called *effectively computable* if a list of instructions can be given that in principle make it possible to determine the value $f(n)$ for any argument n. (This notion extends in an obvious way to two-place and many-place functions.) The instructions must be completely definite and explicit. They should tell you at each step
>
> - what to do,
> - not tell you to go ask someone else what to do,
> - or to figure out for yourself what to do:
>
> the instructions should require no external sources of information, and should require no ingenuity to execute, so that one might hope to automate the process of applying the rules, and have it performed by some mechanical device.

In the next section, we will introduce one notion to make precise this intuitive definition of what it means to be "computable": the *Turing machine*. It will be clear that any Function computable by a Turing machine is also "computable" in the intuitive sense. The converse of this claim – namely, that any intuitively "computable" function can be computed by a Turing machine – is **Turing's Thesis**. We will get back to **Turing's Thesis** (and the **Church-Turing Thesis**) later in the course.

3 Turing computability

3.1 Turing machines

I will explain Turing machines and draw a picture in class. Here is the formal definition:

[2] As we will see later in the course, in the context of "computability" it is completely sufficient to restrict attention to functions of natural numbers. That is because basic arithmetic is, in a certain sense, expressive enough to encapsulate computation.

Definition 3.1 (Turing machine)**.** A *Turing machine (TM)* is a quadruple $T = (Q, \Gamma, \delta, q_0)$, where

- Q is a finite set of *states*;
- Γ is a finite set of *symbols* including a symbol B called "blank";
- $\delta : Q \times \Gamma \rightharpoonup Q \times \Gamma \cup \{L, R\}$ is a partial function called the *transitioning function*; and
- $q_0 \in Q$ is a state called the *initial state*.

A Turing machine operates on a one-dimensional *tape* which is devided into *cells*. On each cell exactly one symbol from Γ is written. The tape extends infinitely in both directions. It doesn't in fact matter whether we think of the tape as in fact *infinite* or imagine that it is finite, but a little man is standing at each end and, whenever the Turing machine tries to move beyond the current end of the tape, glues more cells to the tape end as needed. Only a finite number of cells on the tape contain a symbol other than B – i.e., it is "blank" (or "empty") everywhere except in finitely many places.

We can think of a Turing machine $T = (Q, \Gamma, \delta, q_0)$ operating on the tape as follows: T has a "reading head" which, at each point in time, is positioned on top of one cell of the tape. T is started in its initial state q_0 on some cell. At each time point, it is in some internal state $q \in Q$ and can "read" the symbol written on the cell under its reading head. The action performed by T in one step is specified by the transitioning function δ and is thus determined by the current internal state q and the symbol under its head. At each step, T can perform one of the following actions:

1. If the current state is q and the symbol read is γ, then set the new state to q' and replace γ by γ'.
2. If the current state is q and the symbol read is γ, then set the new state to q' and move to the *L*eft.
3. If the current state is q and the symbol read is γ, then set the new state to q' and move to the *R*ight.

Note that δ is a partial function. This means that for each combination (q, γ) of a state q and a symbol γ there is *at most* one instruction that applies to T in any situation. Thus, a fourth action of T (better described as a non-action) is: do nothing, if there is no instruction that applies. We say that T *halts* when this happens.

3.1.1 Monadic Turing machines

So far, we have defined a Turing machine $T = (Q, \Gamma, \delta, q_0)$, where Γ is any finite "alphabet", or set of symbols that can be written in the cells of the tape that the Turing machine operates on, as long as Γ contains the "blank" symbol B.

For the time being, we will simplify this definition, and only consider *monadic* Turing machines, where $\Gamma = \{1, B\}$. That is, for such a Turing machine, each cell of the tape contains either a stroke (1) or is blank (B). The following discussion is based on monadic Turing machines of this kind.

3.1.2 Specifying Turing machines

For a given TM $T = (Q, \Gamma, \delta, q_0)$, each of the three actions described in the previous section corresponds to an assignment $(q, \gamma) \mapsto (q', a)$ of δ, where $a \in \Gamma \cup \{L, R\}$. Note that by definition of a TM, the set of states Q and the tape alphabet Γ is finite, so the instructions given by δ can be represented as a finite lookup table or as a flowchart. Here is an example of these representations for the same Turing machine T:

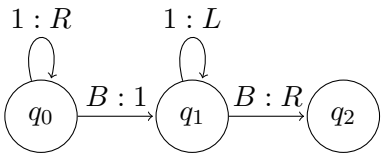

Figure 1: Diagram representation of T

	q_0	q_1	q_2
B	$q_1, 1$	q_2, R	
1	q_0, R	q_1, L	

Figure 2: Table representation of T

In the diagram representation, the circles represent states, and arrows between states representing actions. The arrows are labelled with two symbols separated by a colon (:). The left symbol is an element of Γ, and the right symbol is either an element of Γ, or an element of $\{L, R\}$. In the example above, the arrow from q_0 to q_1 with the label $B : 1$ means: "If in state q_0 and you scan B, then change the state to q_1 and write a 1". The arrow from q_0 to itself with the label $1 : R$ means: "If in state q_0 and you scan 1, then remain in state q_0 and move one cell to the *R*ight".

In the table representation, columns correspond to states Q, and rows correspond to symbols from the alphabet Γ. Each cell of the table contains two symbols: One state, and either one symbol from Γ or an element of $\{L, R\}$. In the example above, the entry $q_1, 1$ in column q_0 and row B means: "If in state q_0 and you scan B, then change the state to q_1 and write a 1".

The lack of an outgoing arrow from a state q with a label whose left symbol is γ, and similarly an empty table cell incolumn q and row γ means, means that $\delta(q, \gamma)$ is undefined.

3.2 Computations of Turing machines

Our goal for this section is to define what computations of Turing machines are, and what it means for a Turing machine to compute a function on the natural numbers.

3.2.1 Configurations

We saw that the next *step* of a Turing machine T is determined by its current state q and the symbol under the tape head γ. The *entire future behavior* of T is determined by:

- the current internal state q;
- the tape content; and
- the position of the reading head on the tape.

Since we assumed that T is started on a tape which contains only a finite number of non-blank symbols, the tape content at each step can be described as a finite string of symbols (from the first to the last non-blank symbol on the tape). Thus, we can represent a *configuration* of T as follows.

Definition 3.2 (Configuration)**.** A *configuration* of a Turing machine $T = (Q, \Gamma, \delta, q_0)$ is a quadruple $(\alpha, q, \gamma, \beta)$, where

- α is the string of symbols to the *left* of the reading head of T starting from the first non-blank symbol of the tape if there *are* non-blank symbols to the left of the reading head or B otherwise;
- β is the string of symbols to the *right* of the reading head of T ending at the last non-blank symbol of the tape if there *are* non-blank symbols to to the right of the reading head or B otherwise;
- $\gamma \in \Gamma$ is the symbol *under* the reading head of T; and
- $q \in Q$ is the current *state* of T.

A convenient way of writing down configurations is to write out the string $\alpha\gamma\beta$ and attach the state q as a subscript to γ. For example, 1101_q0011 denotes the configuration $(110, q, 1, 0011)$.

The next definition tells us when a Turing machine stops operating.

Definition 3.3 (Halting configuration)**.** A configuration $C = (\alpha, q, \gamma, \beta)$ is a *halting configuration* of a TM $T = (Q, \Gamma, \delta, q_0)$ if no instruction of δ applies to C – i.e., if $\delta(q, \gamma)$ is undefined.

For any configuration that is not a halting configuration, when can determine what configuration comes next by checking the instructions in δ.

Definition 3.4 (Next configuration)**.** Given a TM $T = (Q, \Gamma, \delta, q_0)$ and a non-halting configuration $C = (\alpha, q, \gamma, \beta)$, the *next configuration* C' or the *configuration* C' *after* C is determined as follows:

- "Write instruction": If $\delta(q, \gamma) = (q', \gamma')$ for some $q' \in Q$ and $\gamma' \in \Gamma$, then $C' = (\alpha, q', \gamma', \beta)$.
- "Move left instruction": If $\delta(q, \gamma) = (q', L)$ for some $q' \in Q$, then $C' = (\alpha', q', \gamma\beta)$, where α' is all of α except its *last* symbol if the length of α is at least 2, and B otherwise.
- "Move right instruction": If $\delta(q, \gamma) = (q', R)$ for some $q' \in Q$, then $C' = (\alpha\gamma, q', \gamma')$, where γ' is all of γ except its *first* symbol if the length of γ is at least 2, and B otherwise.

3.2.2 Representing inputs and outputs

Almost there! Before we can define what it means for a Turing machine to compute a function, we need some conventions about how we represent inputs and outputs.

The idea is that we represent a natural number n as a block of n strokes (1s) on the tape. However, this makes it somewhat confusing what the representation of 0 is – so it is a better idea to use a block of $n + 1$ strokes to represent n.

If we want to pass more than one input to a TM, we simply separate the blocks corresponding to the inputs by a single blank (B). Thus, the input of the three arguments $3, 0, 2$ is represented by

$$1111B1B111.$$

We represent outputs the same way.

We adopt the convention that, for an input $(x_1, \ldots, x_n)$ of n natural numbers, we represent $(x_1, \ldots, x_n)$ on the tape as described above, and otherwise the tape is empty (blank). In the beginning of its computation, the Turing machine scans the leftmost stroke of the block for x_1, and its state is q_0.

Similarly, we adopt the convention that a configuration of T represents an ouput $y \in \mathbb{N}$ if the tape contains a block of $y + 1$ strokes and is otherwise blank, T is scanning the leftmost stroke of this block, and T is halted – i.e., it is in a state q such that no instruction of δ applies.

The next two definitions capture these conventions formally. For a natural number n, write 1^n for a block of n 1s. For example, $1^3 = 111$.

Definition 3.5 (Initial configuration for an input)**.** Given a monadic TM T and natural numbers $x_1, \ldots, x_n$, the *initial configuration of* T *for input* $x_1, \ldots, x_n$ is the configuration

$$1_{q_0} 1^{x_1} B 1^{x_2+1} B \cdots 1^{x_n+1}.$$

Definition 3.6 (Final configuration for an output)**.** Given a monadic TM T and a natural number y, a *final configuration of T for output y* is a halting configuration

$$1_q 1^y.$$

(Recall 3.3: The configuration $1_q 1^y$ is a halting configuration if $\delta(q, 1)$ is undefined.)

3.2.3 Function computed by a Turing machine

Now we're *really* almost there.

Definition 3.7 (Computation sequence, halting)**.** Let T be a Turing machine and $(x_1, \ldots, x_n)$ an n-tuple of natural numbers. A *partial computation sequence of T for input* $(x_1, \ldots, x_n)$ is a sequence of configurations $C_1, C_2, \ldots, C_k$ such that:

- C_0 is the initial configuration for input $(x_1, \ldots, x_n)$;
- for each $i < k$, C_{i+1} is the configuration after C_i (according to T, see Definition 3.4).

A *halting* computation sequence of T for input $(x_1, \ldots, x_n)$ is a partial computation sequence where the last configuration is a halting configuration.

We say that T *halts on input* $(x_1, \ldots, x_n)$ iff there exists a halting computation of T for input $(x_1, \ldots, x_n)$.

Finally!

Definition 3.8 (Partial function computed by a TM)**.** Let T be a monadic TM and f an n-ary *partial* function from $\mathbb{N}$ to $\mathbb{N}$. We say that T *computes the partial function f* if the following holds:

For every n-tuple of natural numbers $(x_1, \ldots, x_n)$, if T is started in the initial configuration for $(x_1, \ldots, x_n)$, then

- if $f(x_1, \ldots, x_n)$ is defined and $f(x_1, \ldots, x_n) = y$, then T halts in a final configuration for y;
- if $f(x_1, \ldots, x_n)$ is undefined, then T either does not halt or it halts in a non-final configuration for y.

Every Turing machine T computes some (partial) function. We write f_T for *the function f_T computed by T*.

A TM T computing a partial function f started on the representation of an input for which f is undefined either:

- does not halt, i.e. it "loops" forever; or

- it does halt, but then it halts in a non-final configuration. According to Definition 3.6, this is the case when one of the following happens:
  - T halts on an entirely empty tape;
  - T halts on a tape that contains more than one consecutive block of 1s; or
  - T halts and is scanning a 1 that is not the leftmost 1 on the tape.

The following definition is just a special case of Definition 3.8.

Definition 3.9 (Total function computed by a TM)**.** Let T be a monadic TM and f an n-ary *total* function from $\mathbb{N}$ to $\mathbb{N}$. We say that T *computes the total function* f if:

For every n-tuple of natural numbers $(x_1, \ldots, x_n)$, if T is started in the initial configuration for $(x_1, \ldots, x_n)$, then T halts in a final configuration for $f(x_1, \ldots, x_n)$.

Definition 3.10. An n-ary total or partial function from $\mathbb{N}$ to $\mathbb{N}$ is *Turing-computable* if there exists a Turing machine that computes it.

Theorem 3.11. *The following functions are Turing computable:*

1. $f(x) = x + 1$
2. $g(x, y) = x + y$
3. $h(x, y) = x \times y$

Proof. Exercise. We will do one of them in class. □

3.3 Turing's thesis

If you have seen definitions of Turing machines before, they might have been different from the one I gave above. Here are some common variations:

- Allow an arbitrary finite alphabet Γ instead of just the two symbols 1 and B.
- TMs with a total instead of a partial transitioning function δ with a designated set of *halting states*.
- Tape infinite in only one direction.
- TMs that can read and write simultaneously on more than one tape.
- TMs that operate on a two-dimensional grid rather than a one-dimensional tape.
- Nondeterministic TMs that allow for more than one applicable instruction at each step. Here, the TM can nondeterministically "choose" what to do next.

It turns out that all these different ways of defining a Turing machine are invariant with respect to the class of Turing computable functions – that is, one can prove that the examples I gave above all compute *the same functions*.

(**Mini-excursion into complexity theory.** There *are* complexity-related differences between different ways of defining Turing machines: The above variations of Turing machines differ in how much time or memory they need to compute certain functions. In fact, the famous P-vs-NP problem of theoretical computer science essentially boils down to different definitions of a Turing machine:

P is defined as the class of decision problems that can be solved by a deterministic Turing machine (just like the one given in Definition 3.1) in polynomial time relative to input size. NP is defined as the class of decision problems that can be solved by a *non*-deterministic Turing machine in polynomial time relative to input size. It is an open question whether or not P = NP. Thankfully, in computability theory, we do not need to care about time or space complexity, so for us deterministic and nondeterministic Turing machines are equivalent.)

It is not too hard to prove that all the above variations of Turing machine compute the same functions.

It should also be clear that every Turing-computable function is *effectively computable* in the sense of Section 2: What a TM does at each step is determined by a finite list of instructions. I should be able (on a good day) to simulate the actions of a Turing machine, and it doesn't require much coding competence to write a program in, for example, Python, that does the same.

The converse claim that *any effectively computable function* as discussed in Section 2 is Turing computable called *Turing's Thesis*.

> **Turing's Thesis.** *Any function that is effectively computable (computable by an algorithm) is Turing-computable.*

Note that this is a bold claim: While we have seen that addition and multiplication of natural numbers are Turing-computable, Turing's Thesis implies that for *any* function that we intuitively deem computable at all there exists a Turing machine that computes it.

Turing's thesis cannot be directly proved, because it is a claim that involves the intuitive, unformalized notion of *effective* (or *intuitive*) computability. There are, however, arguments that can be given for it. We will return to this issue in the section on the Church-Turing thesis.

Problems

1. Prove that $f(x) = x + 1$ is Turing-computable.
2. Prove that $g(x, y) = x + y$ is Turing-computable.
3. Prove that $h(x, y) = x \times y$ is Turing-computable.

4. Prove that $i(x, y) = \min(x, y)$ is Turing-computable.

5. Prove that $j(x, y) = \max(x, y)$ is Turing-computable.

4 Enumeration and diagonalization

4.1 Preliminaries

In this section, we will make frequent use of the following concepts and facts. If you are unfamiliar with them, it would be a good idea to brush up before continuing to read this section.

Let X, Y be two sets. A function $f : X \to Y$ is...

- *injective* if, for all $x \in X$ and $y \in Y$, $x \neq y \implies f(x) \neq f(y)$.
- *surjective* if, for all $y \in Y$ there exists $x \in X$ such that $f(x) = y$.
- *bijective* if it is injective and surjective. In that case each $x \in X$ corresponds to exactly one $y \in Y$ and vice versa. We say that X and Y have the same *cardinality*.

For any two sets X, Y, there exists an injection $f : X \to Y$ if and only if there exists a surjection $g : Y \to X$. Moreover, for any bijection $f : X \to Y$, its *inverse* $f^{-1} : Y \to X$ defined by

$$f^{-1}(y) = \text{ the unique } x \in X \text{ s.t. } f(x) = y$$

is also a bijection.

Let X, Y, Z be three sets, and $f : X \to Y$ and $g : Y \to Z$ functions. The composition $g \circ f : X \to Z$ is defined by $(g \circ f)(x) = g(f(x))$.

- If f, g are both injective, then so is $g \circ f$.
- If f, g are both surjective, then so is $g \circ f$.
- If f, g are both bijective, then so is $g \circ f$.

4.2 Countable sets

There are small sets and there are big sets. In fact, there are even very, very big sets. Of course, there are finite sets, like $\{1, 2, 3\}$. Then there are infinite sets like the set $\mathbb{N}$ of all natural numbers, or the set $\{0, 2, 4, \ldots\}$ of all even numbers, or the set $\mathbb{R}$ of all real numbers.

Definition 4.1 (Finite and infinite sets)**.** A set S is *finite* if there exists a bijection $f : \{1, 2, \ldots, n\} \to S$ for some $n \in \mathbb{N}$.

S is *infinite* if it is not finite.

Infinite sets are somewhat strange. Consider the following example:

Hilbert's paradox of the Grand Hotel:

Imagine there is a hotel with infinitely many rooms, numbered $1, 2, 3, \ldots$. Suppose the hotel is full: each room is occupied by a guest. Now a finite number of guests arrives - let's say, 3. The hotel manager is clever and figures out a way of accomodating the newcomers without kicking anyone out: he simply asks each guest to move up three rooms. That is, if a guest currently occupies room n, she is asked to move to room $n + 3$. And thus the three newcomers get their rooms and no one has to sleep under the bridge!

But what if an infinity of new guests arrives? Let's say we have one newcomer for each natural number – i.e., the newcomers are $1, 2, 3, \ldots$. The hotel manager turns out to be even more clever than we thought – in fact, he turns out to be a logician. He asks the guest in room 1 to move to room 2, the guest in room 2 to room 4, and in general the guest in room n to move to room $2n$. Lo and behold, all the odd-numbered rooms became available – enough to accomodate all the newcomers.

The clever logician-turned-hotel-manager's trick only worked because he was lucky and only a *countable* number of newcomers arrived – that is, we could *enumerate* the new guests by $1, 2, 3, \ldots$. If a much larger number of guests had arrived – namely, more than we can enumerate by natural numbers – the poor hotel manager would not have been able to accomodate them all. Let's make this precise.

Definition 4.2 (Countable set)**.** A set S is *countable* if it is finite[3] or there exists a surjective function $f : \mathbb{N} \to S$.

(An equivalent definition is: S is *countable* if it is finite or there exists an injective function $g : S \to \mathbb{N}$.)

The existence of a surjective function from $\mathbb{N}$ to a set S means that each element of S is "hit" by some natural number – just as it was the case with the infinite set of newcomers to Hilbert's Hotel. Note that, according to this definition, every *finite* set is also countable. In this course, we will mostly deal with *countably infinite sets*, which are, unsurprisingly, sets that are both infinite and countable. We will often use the following characterization:

Proposition 4.3 (Countably infinite set)**.** *A set S is countably infinite if and only if there exists a bijection $f : \mathbb{N} \to S$.*

The intuition for countably infinite sets is that they all have the same cardinality, or size, as the natural numbers. If S is countably infinite, then there is a bijection

[3] Technically, it would suffice to require that S is empty or there exists a surjection from $\mathbb{N}$ to S.

$f : \mathbb{N} \to S$, and so we can enumerate all elements of S by

$$f(1), f(2), f(3), \ldots.$$

Let's see some examples of countably infinite sets.

Proposition 4.4. *The following sets are all countable:*

1. *The set* $\mathbb{Z}$ *of integers.*
2. *The set* $\mathbb{N} \times \mathbb{N}$ *of pairs of natural numbers.*
3. *The set* $\{q \in \mathbb{Q} \mid Q \geq 0\}$ *of positive rational numbers.*
4. *The set* $\mathbb{N} \times (\mathbb{N} \times \mathbb{N})$ *of triples of natural numbers.*
5. *The set of* $\mathbb{Q}$ *all rational numbers.*
6. *The set* $\{(n_1, \ldots, n_k) \mid k \in \mathbb{N} \wedge n_i \in \mathbb{N}\}$ *of all finite sequences of natural numbers.*
7. *The set of all* finite *subsets of* $\mathbb{N}$.

Proof. We'll do some in class, and I leave the rest as exercises. I recommend you do them in the given order. □

Proposition 4.5. *Let* S *and* T *be sets. If* S *is countably infinite and there exists a bijection* $g : S \to T$*, then* T *is also countably infinite.*

Proof. S is countably infinite, so there is a bijection $f : \mathbb{N} \to S$. Since $g : S \to T$ is a bijection, and the composition of two bijections is also a bijection, $g \circ f$ is a bijection from $\mathbb{N}$ to T. Therefore, T is countably infinite. □

Proposition 4.6. 1. *Any subset of a countable set is also countable.*

2. *Any* infinite *subset of a countably* infinite *set is also countably infinite.*

Proof. Let T be a countable set and $S \subseteq T$.

- If S is finite, then it's countable, and we're done.
- If S is infinite, then T must be infinite as well. But T is countable, so there exists a bijection $f : \mathbb{N} \to T$. We need to show that S is countably infinite as well. Construct a bijection $g : \mathbb{N} \to S$ as follows:

  Enumerate the elements of T using f: $f(1), f(2), \ldots$.

  Let n_1 be the smallest natural number such that $f(n_1) \in S$. Set $g(1) := f(n_1)$.

Suppose we have already defined $g(1), \ldots, g(k)$. Let n_{k+1} be the smallest natural number with $n_{k+1} > n_k$ such that $f(n_{k+1}) \in S$. Set $g(k+1) := f(n_{k+1})$.

Then by construction, $g : \mathbb{N} \to S$ is injective since each element of S appears only once, and it is surjective since each element of S appears somewhere in the sequence $f(1), f(2), \ldots$, so it will be picked eventually. Therefore, g is bijective, and so S is indeed countably infinite.

□

So there are finite sets and there are countably infinite sets. What about *un*countably infinite sets? We will encounter those in the next section.

4.3 Diagonalization

Theorem 4.7 (Cantor's Theorem)**.** *The set $\mathcal{P}(\mathbb{N})$ of all sets of natural numbers is uncountable.*

Proof. Suppose towards a contradiction that $\mathcal{P}(\mathbb{N})$ is countable. Then there exists a bijection $f : \mathbb{N} \to \mathcal{P}(\mathbb{N})$. Using f, we can enumerate all sets of natural numbers by $f(1), f(2), \ldots$. Given any set S of natural numbers, we can represent it using its characteristic function $\chi_S : \mathbb{N} \to S$, which is defined as

$$\chi_S(n) = \begin{cases} 1 \text{ if } n \in S; \\ 0 \text{ if } n \notin S. \end{cases}$$

That is, χ_S tells us for each $n \in \mathbb{N}$ whether or not $n \in S$.

Now we can represent $\mathcal{P}(\mathbb{N})$ as an infinite two-dimensional array, where:

- Each *column* i corresponds to the natural number i.
- Each *row* j corresponds to the set $f(j)$ of natural numbers from our enumeration using f, represented by its characteristic function $\chi_{f(j)} : \mathbb{N} \to \{0, 1\}$.
- Each *cell* in column i and row j contains the value of $\chi_{f(j)}(i)$, i.e. 0 if $i \in f(j)$ and 1 if $i \notin f(j)$.

It looks like this:

	1	2	3	4	$\cdots$
$\chi_{f(1)}$	$\chi_{f(1)}(1)$	$\chi_{f(1)}(2)$	$\chi_{f(1)}(3)$	$\chi_{f(1)}(4)$	$\cdots$
$\chi_{f(2)}$	$\chi_{f(2)}(1)$	$\chi_{f(2)}(2)$	$\chi_{f(2)}(3)$	$\chi_{f(2)}(4)$	$\cdots$
$\chi_{f(3)}$	$\chi_{f(3)}(1)$	$\chi_{f(3)}(2)$	$\chi_{f(3)}(3)$	$\chi_{f(3)}(4)$	$\cdots$
$\chi_{f(4)}$	$\chi_{f(4)}(1)$	$\chi_{f(4)}(2)$	$\chi_{f(4)}(3)$	$\chi_{f(4)}(4)$	$\cdots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$

Now consider the *diagonal sequence* of values in the diagonal (top-left to bottom-right) of this array, i.e. the sequence

$$\chi_{f(1)}(1),\ \chi_{f(2)}(2),\ \chi_{f(3)}(3),\ \ldots$$

This is an infinite sequence of 0s and 1s.

Now define the *antidiagonal sequence* $\delta : \mathbb{N} \to \{0, 1\}$ by flipping all the 0s and 1s in the diagonal sequence, i.e.

$$\delta(n) := 1 - \chi_{f(n)}(n) \text{ for each } n \in \mathbb{N}.$$

Then δ is a characteristic function of a set of natural numbers – that is, we can define the set $\Delta \subseteq \mathbb{N}$ by

$$\Delta := \{n \in \mathbb{N} \mid \delta(n) = 1\}.$$

By our assumption that f enumerates all sets of natural numbers, since Δ is one of them, we have $\Delta = f(m)$ for some $m \in \mathbb{N}$. But that means that $\chi_{f(m)} = \delta$ – i.e., the mth row of the array above is exactly the characteristic function of Δ. Then the value in the cell of row m and column m is $\delta(m)$. By definition of δ, we have

$$\delta(m) = 1 - \chi_{f(m)}(m) = 1 - \delta(m). \tag{1}$$

But that's impossible: We know that $\chi_{f(m)}(m)$ equals either 0 or 1. If $\chi_{f(m)}(m) = 0$, then by (1) we have $\delta(m) = 1 = 0$. And if $\chi_{f(m)}(m) = 1$, then by (1) we have $\delta(m) = 0 = 1$. In both cases we have a contradiction, so our initial assumption that $\mathcal{P}(\mathbb{N})$ is countable must be false. Therefore, the set of all sets of natural numbers is uncountable indeed. □

Corollary 4.8. *The set of real numbers is uncountable.*

Proof. Exercise. □

Corollary 4.9. *The set $\{f : \mathbb{N} \to \mathbb{N}\}$ of all unary functions of natural numbers is uncountable.*

Proof. Let

$$\mathcal{B} := \{f : \mathbb{N} \to \{0, 1\}\}$$

be the set of all unary functions on natural numbers whose values are only 0 or 1. $\mathcal{B}$ is the set of all characteristic functions on $\mathbb{N}$, and each $f \in \mathcal{B}$ corresponds to exactly one subset of $\mathbb{N}$, namely

$$S_f := \{n \in \mathbb{N} \mid f(n) = 1\}.$$

Conversely, every subset $S \subseteq \mathbb{N}$ determines a function

$$f_S(n) := \begin{cases} 1 \text{ if } n \in S; \\ 0 \text{ if } n \notin S. \end{cases}$$

Thus, the mapping $f \mapsto f_S$ is a bijection from $\mathcal{B}$ to $\mathcal{P}(\mathbb{N})$. Since $\mathcal{P}(\mathbb{N})$ is uncountable by Cantor's Theorem, it follows by Proposition 4.5 that $\mathcal{B}$ is also uncountable. And since $\mathcal{B} \subseteq \{f : \mathbb{N} \to \mathbb{N}\}$, it follows from Proposition 4.6 that $\{f : \mathbb{N} \to \mathbb{N}\}$ is uncountable as well. □

4.4 Enumeration of Turing machines

Let $T = (Q, \Gamma, \delta, q_0)$ be a monadic Turing machine with $\Gamma = \{B, 1\}$. Suppose $Q = \{q_1, \ldots, q_n\}$.

From T, construct a TM $T' = (Q', \Gamma, \delta', q_0)$, where $\delta : Q \times \Gamma \to Q \times \Gamma \cup \{L, R\}$ is a *total* function as follows:

- Take $Q' := \{q_1, \ldots, q_n, q_{n+1}\}$, i.e., add an extra state to Q. This extra state will be the "halting" state.
- Define $\delta' : Q' \times \Gamma \to Q' \times \Gamma \cup \{L, R\}$ such that

$$\delta'(q, \gamma) := \begin{cases} (q', a) & \text{if } q \in Q \text{ and } \delta(q, \gamma) \text{ defined and } \delta(q, \gamma) = (q', a); \\ (q_{n+1}, \gamma) & \text{if } q \in Q \text{ and } \delta(q, \gamma) \text{ undefined;} \\ (q_{n+1}, \gamma) & \text{if } q = q_{n+1}. \end{cases}$$

Then T' computes the same function as T and has the extra property that δ' is total. Represented as a flowchart diagram, this means that each state of T' has exactly two outgoing arrows: one for 1 and one for B. And represented as a table, this means that we have $n+1$ columns (one for each state in Q'), 2 rows (one for 1 and one for B, and *each* cell contains a pair (q', a), where $q' \in Q'$ and $a \in \Gamma \cup \{L, R\}$.

For example, the table representation of T' might look like this:

	q_0	q_1	q_2
B	$(q_1, 1)$	(q_2, R)	(q_2, B)
1	(q_0, R)	(q_1, L)	$(q_2, 1)$

We can enumerate the cells in this table using the "zig-zag" method. In the example above, we enumerate the content of the table cells corresponding to the (column, row) combinations

$$(q_0, B), (q_0, 1), (q_1, B), (q_1, 1), (q_2, B), (q_2, 1)$$

so we get the enumeration

$$(q_1, 1), (q_0, R), (q_2, R), (q_1, L), (q_2, B), (q_2, 1).$$

Now we can simply write $k+1$ for each state $q_k \in Q'$, 1 for stroke (ha, it's already 1!), 2 for B, 3 for L, and 4 for R. Thus, for the example above the numeration becomes:

$$(2, 1), (1, 4), (3, 4), (2, 3), (3, 2), (3, 1)$$

We can also just drop the parentheses, since it is clear that each number in an odd position stands for a state and each number in an even position stands for an element of $\{B, 1, L, R\}$. So we can represent T' as a finite sequence of natural numbers of length $2(n+1)$.

We can even represent T' as a single natural number by using prime decomposition. For the example above:

$$2^2 \cdot 3^1 \cdot 5^1 \cdot 7^4 \cdot 11^3 \cdot 13^4 \cdot 17^2 \cdot 19^3 \cdot 23^3 \cdot 29^2 \cdot 31^3 \cdot 37^1$$

So the Turing machine T' is represented by the number 1224390158628055517981801624105 40.

In summary, we have shown how we can represent any Turing machine T by a natural number. The procedure above gives a mapping $T \mapsto n_T$ which assigns a natural number n_T to any Turing machine T. Let's call this mapping g, then $g : \mathsf{TM} \to \mathsf{TCF}$ is a function from the set TM of Turing machines to the set TCF of Turing-computable partial functions on the natural numbers. This function g is injective, because any variation of the definition of T yields a different table representing T, which yields a different sequence, which yields a different natural number. So $T \neq T' \implies n_T \neq n_{T'}$.

By Definition 4.2, a set S is countable if it is finite or there exists an injective function from S to $\mathbb{N}$. Clearly TM is not finite. We constructed an injection from TM to $\mathbb{N}$, so we have proved that TM is countable.

Theorem 4.10. *The* TM *set of Turing machines is countable.*

Proof. The construction above defines an injection from the infinite set TM to $\mathbb{N}$. □

Note that our function g assigning numbers to Turing machines "leaves out" a lot of numbers: The "smallest" Turing machine (in terms of the number representing it) with a total transitioning function δ consists of one state q_0 with δ as follows:

	q_0
B	$(q_0, 1)$
1	$(q_0, 1)$

This TM is represented by the number

$$2^1 \cdot 3^1 \cdot 5^1 \cdot 7^1 = 210.$$

So none of the natural numbers less than 210 represent Turing machines! This is quite a "gappy" enumeration. In other words, our function $g : \mathsf{TM} \to \mathbb{N}$ is injective, but not surjective.

But we can do better: A nice feature of our construction above is that it is "reversible": for a given number n, we can check whether it represents a Turing machine, and if it does, reconstruct that Turing machine. This allows us to construct a bijection $g' : \mathbb{N} \to \mathsf{TM}$ as follows: we iterate through the natural numbers and, for

each n, check whether n represents a Turing machine T. If it does, we add T to our enumeration g'. Otherwise, we continue looking for the next number representing a Turing machine. Since there are infinitely many Turing machines, at each stage of the construction we are guaranteed to find a next number that corresponds to a Turing machine.

This procedure amounts to "closing the gaps" that were left by g, and g' is a complete and non-gappy enumeration

$$g'(1), g'(2), g'(3), \ldots$$

of all Turing machines.

Now $g' : \mathbb{N} \to \mathsf{TM}$ is

- *injective* because different numbers represent different Turing machines; and it is
- *surjective* because every Turing machine is represented by some number.

So we have a bijection $g' : \mathbb{N} \to \mathsf{TM}$.

Now every Turing machine T computes a partial function f_T on the natural numbers. Thus, the mapping $T \mapsto f_T$ that assigns to a Turing machine T the function f_T computed by T defines a function $f : \mathsf{TM} \to \mathsf{TCF}$ from the set TM of Turing machines to the set TCF of Turing-computable partial functions on the natural numbers. This function f is not injective, because different Turing machines may compute the same function, but it is surjective, since by definition of "Turing-computable" every function $f \in \mathsf{TCF}$ is computed by *some* Turing machine $T \in \mathsf{TM}$.

Now the composition $f \circ g'$ is a function from $\mathbb{N}$ to TCF. Since both f and g' are surjective, $f \circ g'$ is surjective. And clearly TCF is infinite, so we proved:

Corollary 4.11. *The set* TCF *of partial Turing-computable functions is countable.*

Note that each f_k in our enumeration may be a unary, or a binary, or in general an n-ary function on the natural numbers for any $n \in \mathbb{N}$.

Note also that the same function f is computed by multiple Turing machines. In fact, every function is computed by *infinitely many* Turing machines: Given a TM T that computes f, you can construct $T', T'', ...$ by adding one, or two, or ... "unreachable" states. This does not alter the behavior of T at all, and so $T, T', T'', ...$ all compute f.

Thus, although we could easily "close" the gaps of our enumeration of Turing machines and of Turing-computable functions, that enumeration is redundant. Could we do better and make sure that we mention each Turing-computable function exactly once in our infinite list? It turns out we cannot – this is a consequence of Rice's Theorem, which we will prove later in the course.

Remark. In the following, I will sometimes write "the n-th Turing machine" (or simply T_n) and "the n-th Turing-computable function" (or simply f_n), implicitly referrung to the enumeration constructed above.

4.5 Existence of uncomputable functions: counting argument

In the previous section, we saw that there is an enumeration

$$f_1, f_2, \ldots$$

of all Turing-computable functions. This means there are "as many" Turing-computable functions as there are natural numbers. (Our enumeration mentions each function infinitely many times, but a moment of thought shows that there are also infinitely many different Turing-computable functions.)

However, we also saw (Corollary 4.9) that the set $\{f : \mathbb{N} \to \mathbb{N}\}$ of all unary functions on the natural numbers is *un*countable. Thus, there are "more" such functions than there are natural numbers.

Taken together, these two facts imply that *there are more functions than there are Turing-computable functions.* So there must exist functions that are not Turing-computable!

Theorem 4.12. *There are functions on the natural numbers that are not Turing-computable.*

Assuming Turing's Thesis, this means that there are functions (on natural numbers) that are uncomputable by any algorithm or effective procedure whatsoever.

Note how we proved the existence of uncomputable functions: we showed that there are countably many *computable* functions (on the natural numbers, of any finite arity), and we showed that there are more – namely, *uncomputably* many – (unary) functions on the natural numbers overall. Thus, there are strictly *more* functions on the natural numbers than there are *computable* functions on the natural numbers. Therefore, there must be uncomputable functions. This kind of cardinality-based argument is called a *counting argument.* While it is very elegant, it can feel a bit like cheating. After all, we haven't given any specific uncomputable function! So we will do that next.

4.6 Existence of uncomputable functions: the diagonal function

The following construction is essentially analogous to the *antidiagonal sequence* that we constructed in the proof of Theorem 4.7 (Cantor's Theorem).

Let $f_1, f_2, \ldots$ be the enumeration of Turing-computable functions on the natural numbers from Corollary 4.11.

Definition 4.13. Define the *diagonal function* $d : \mathbb{N} \to \mathbb{N}$ as follows:

$$d(n) := \begin{cases} 2 & \text{if } f_n(n) \text{ is defined and } f_n(n) = 1; \\ 1 & \text{otherwise.} \end{cases}$$

This function is the promised Turing-uncomputable function!

Theorem 4.14. *The diagonal function d is not Turing-computable, and thus (assuming Turing's Thesis) also not effectively computable.*

Proof. Suppose towards a contradiction that d is Turing-computable. Then d occurs somewhere in the enumeration $f_1, f_2, \ldots$ of Turing-computable functions – let's say $d = f_m$. Then, for all n, either $d(n)$ and $f_m(n)$ are both undefined, or they're both defined and equal. But now consider the case $n = m$:

- If $f_m(m)$ is undefined, then by definition of d, we have $d(m) = 1$. But then $f_m(m) = d(m) = 1$. Contradiction.
- If $f_m(m)$ is defined, then $f_m(m) = k$ for some $k \in \mathbb{N}$.
  - If $k = 1$, then by definition of d, we have $d(m) = 2$. But then $f_m(m) = d(m) = 2$. Contradiction.
  - If $k \neq 1$, then by definition of d, we have $d(m) = 1$. But then $f_m(m) = d(m) = 1$. Contradiction.

So it cannot be true that d is Turing-computable after all. □

By Turing's Thesis, d is a function on the natural numbers that cannot be computed by *any* Turing machine, or more generally by *any* algorithm whatsoever.

You may be thinking right now: wait– we gave a precise definition of the diagonal function d in Definition 4.13. In that definition, we refer to our enumeration $f_1, f_2, \ldots$ of Turing-computable functions, but we *also* gave a perfectly precise definition of that by means of the construction in Section 4.4. So, can't I just *manually* compute the value of $d(n)$ for any n as follows: Given n,

1. reconstruct the n-th Turing machine T_n in our enumeration; then T_n computes f_n;
2. given the specification of T_n (e.g., as a flowchart diagram), figure out what T_n does on input n;
3. if T_n halts on input n with output y, then I know that $f_n(n)$ is defined and $f_n(n) = y$. And if $y = 1$, I know that $d(n) = 2$;
4. if T_n halts on input n with output y and $y \neq 1$, then I know that $d(n) = 1$;
5. and finally, if T_n doesn't halt on input n, or it halts but in a configuration that is not a final configuration for any output, then I know that $f_n(n)$ is undefined, and so I also know that $d(n) = 1$.

Isn't this an algorithm that computes d? If it was, then d would be effectively computable (and, assuming Turing's Thesis, Turing-computable).

Try to imagine actually following the rules for this algorithm given above. You can in fact carry out all instructions for a given n *as long as* T_n *halts on input* n. But what if T_n doesn't halt? You would "simulate" the computation of T_n by figuring out step-by-step what it does next, and it keeps running. You *never know whether* T_n *is done with its computation yet!* The problem is that, if T_n halts on input n, you do not know how many steps it will perform before it halts. So, as long as T_n is still running, it could either

- still halt at some point in the future; *or*
- keep running forever and never halt.

That is precisely the reason why the list of instructions above does *not* give an algorithm, because you are not guaranteed to determine the output after a finite number of steps for any input. No Turing machine can compute T. And, if Turing's Thesis is true, then no algorithm whatsoever can compute d.

This is a very important concept in computability theory, and we will keep returning to it.

5 The halting problem

We saw at the end of the previous section that the reason why can't compute the value of the diagonal function d for an arbitrary input is that, when we observe a Turing machine performing operations on an input, as long as the Turing machine is still working, we do not know whether it will *eventually* halt or not.

The **Halting Problem** is the task of deciding for any given Turing machine T and an input n whether or not T will halt on input n. No Turing machine can solve the halting problem.

Solving the halting problem amounts to the same as computing the function given in the next definition.

Definition 5.1 (Halting function)**.** The *halting function* is the binary function $h : \mathbb{N} \times \mathbb{N} \to \mathbb{N}$ defined as follows:

$$h(m,n) := \begin{cases} 0 & \text{if TM } T_m \text{ halts on input } n; \\ 1 & \text{otherwise.} \end{cases}$$

Theorem 5.2. *The halting function h is not Turing-computable.*

Proof. We first construct the following two Turing machines:

Copying machine C: Let C be a TM that, started scanning the leftmost stroke on a block of n strokes on an otherwise blank tape, halts scanning the leftmost stroke of a block of n strokes, followed by one B, followed by a second block of n strokes. (I leave it up to you to figure out how to construct such a machine.)

Dithering machine D: Let D be the TM given by the flowchart diagram below.

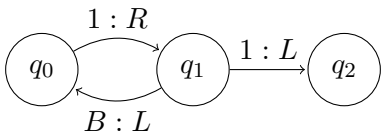

Figure 3: Diagram for D

Started scanning the leftmost stroke of a block of n strokes on an otherwise tape, D eventually halts if $n > 1$, but never halts if $n = 1$. (It's called a *dithering* machine because on an input of $n = 1$ strokes it keeps "dithering" back and forth forever between states q_0 and q_1.)

Now suppose towards a contradiction that the halting function h is Turing-computable. Then there exists a Turing machine T that computes h.

We construct a Turing machine G that computes the function $g(n) = h(n, n)$ by "combining" the Turing machines H and C as follows (what follows is sort of a "proof by picture", but it should be clear how to flesh out all the details formally): If H has k states, denote them by $q_0, \ldots, q_{k-1}$; if C has m states, denote them by $r_0, \ldots, r_{m-1}$. Draw the diagram for C on the left, using the node labels $q_0, \ldots, q_{k-1}$, and draw to its right the diagram for H, using the node labels $r_0, \ldots, r_{m-1}$. Now combine the two diagrams by adding instructions such that, whenever C according to its instructions would halt, the new "combined" machine G instead changes its state to r_0, but doesn't move and doesn't write anything. (This can be done by adding an arrow labelled "$\gamma : \gamma$" from q to r_0 for each state (node) q of C and each $\gamma \in \{B, 1\}$ such that q does not have an outgoing arrow with a label starting with γ.) Then G first executes the instructions of C, and then (when C would have halted) executes the instructions of H. So G computes the function $g(n) = h(n, n)$.

Now we combine G with D in the same way to construct a Turing machine E that first executes the instructions of G, and then those of D. On what inputs does E halt?

E halts if the output of G is a block of $n > 1$ strokes, i.e. if $g(n) > 0$ (remember that we "encode" a number j by a block of $j + 1$ strokes on the tape). And $g(n) > 0$ iff $h(n, n) > 0$ iff TM T_n does not halt on input n.

On the other hand, E does *not* halt if the output of G is a single stroke, i.e. if $g(n) = 0$, which is the case iff $h(n, n) = 0$ iff TM T_n halts on input n.

Now E is a Turing machine, so it occurs somewhere in our enumeration of all Turing machines, say $E = T_j$. Does E halt on input j (i.e., a block of $j + 1$ strokes)?

Suppose E does halt: then, by the second-to-last paragraph above, T_j does *not* halt on input j. But $T_j = E$, contradiction. On the other hand, if E doesn't halt, then by the last paragraph above, $T_j = E$ does halt. We again have a contradiction.

So there can't be a Turing machine that computes h after all, and so h is not Turing-computable. □

So there is no Turing machine that solves the halting problem. Assuming Turing's Thesis, this implies that there is *no effective procedure* or *no algorithm* that solves the halting problem. This means that there is no algorithmic procedure that one can use to "read off" of the specification of a Turing machine as, say, a flowchart diagram, whether it halts on an arbitrary input. In a way, that is to say there the behavior of a Turing machine is incompressible: *the only way to figure out how it behaves on an input is to actually run it on that input.*

Thus, if Turing's Thesis is true, and you figure out a way to tell correctly and in a finite amount of time whether an arbitrary given Turing machine halts on an arbitrary input, then you have solved a computationally unsolvable problem, and your brain has supercomputational powers!

6 Recursive functions

6.1 Primitive recursive functions

You are already familiar with *primitive recurive functions* from the first half of this course. I will restate the definition below, and also introduce the very nice notation for defining primitive recursive functions from [BBJ].

Definition 6.1 (Basic functions)**.** There are three kinds of *basic functions* on $\mathbb{N}$:

1. the unary *zero function* $z : \mathbb{N} \to \mathbb{N}$ with $z(n) = 0$;
2. the unary *successor function* $s : \mathbb{N} \to \mathbb{N}$ with $s(n) = n + 1$;
3. a k-ary *projection function* $p_i^k : \mathbb{N}^k \to \mathbb{N}$ for each k and $i \leq k$ with

$$p_i^k(n_1, \ldots, n_i, \ldots, n_k) = n_i.$$

Definition 6.2 (Composition)**.** Given a k-ary function $f : \mathbb{N}^k \to \mathbb{N}$ and k n-ary functions $g_1, \ldots, g_k : \mathbb{N}^n \to \mathbb{N}$, the *composition* of f with $g_1, \ldots, g_k$, written $\mathrm{Cn}[f, g_1, \ldots, g_k]$, is the n-ary function $h : \mathbb{N}^n \to N$ such that

$$h(x_1, \ldots, x_n) = f(g_1(x_1, \ldots, x_n), \ldots, g_k(x_1, \ldots, x_n)).$$

Definition 6.3 (Primitive recursion)**.** Given an n-ary function $f : \mathbb{N}^n \to \mathbb{N}$ and an $(n+2)$-ary function $g : \mathbb{N}^{n+2} \to \mathbb{N}$, the function obtained by *primitive recursion* from f and g, written $\Pr[f, g]$, is the $(n+1)$-ary function $h : \mathbb{N}^{n+1} \to \mathbb{N}$ such that

$$\begin{aligned} h(x_1, \ldots, x_n, 0) &= f(x_1, \ldots, x_n); \\ h(x_1, \ldots, x_n, s(y)) &= g(x_1, \ldots, x_n, y, h(x_1, \ldots, x_n, y)). \end{aligned}$$

Definition 6.4 (Primitive recursive functions)**.** The set of *primitive recursive functions* is the smallest set that contains the basic functions z, s, and p_i^n (for all $n \in \mathbb{N}$ and $1 \leq i \leq n$) and that is closed under composition and primitive recursion.

Example 6.1. The following are all primitive recursive functions[4]:

1. *Addition:* The binary function $+ : \mathbb{N} \times \mathbb{N} \to \mathbb{N}$ is defined by the equations

$$\begin{aligned} x + 0 &= x \\ x + s(y) &= s(x + y) \end{aligned}$$

   so $+ = \Pr[p_1^1, \mathrm{Cn}[s, p_3^3]]$.

2. *Multiplication:* The binary function $\times : \mathbb{N} \times \mathbb{N} \to \mathbb{N}$ is defined by the equations

$$\begin{aligned} x \times 0 &= 0 \\ x \times s(y) &= x + x \times y \end{aligned}$$

   so $\times = \Pr[z, \mathrm{Cn}[+, p_1^3, p_3^3]]$.

3. *Exponentiation:* Write $\exp(x, y)$ for x^y. Then the binary function $\exp : \mathbb{N} \times \mathbb{N} \to \mathbb{N}$ is defined by the equations

$$\begin{aligned} \exp(x, 0) &= 1 \\ \exp(x, s(y)) &= x \times \exp(x, y) \end{aligned}$$

   so $\exp = \Pr[\mathrm{Cn}[s, z], \mathrm{Cn}[\times, p_1^3, p_3^3]]$.

4. *Predecessor:* $\mathrm{pred} : \mathbb{N} \to \mathbb{N}$ defined by

$$\mathrm{pred}(x) = \begin{cases} 0 & \text{if } x = 0; \\ x - 1 & \text{otherwise.} \end{cases}$$

   (Exercise.)

[4] It is not at all obvious at first sight how to arrive at the definitions using the notation with Cn and Pr. I will explain some in class. If you are confused, you can find more details in Chapter 6 of [BBJ].

5. *Factorial function:* The unary function $! : \mathbb{N} \to \mathbb{N}$ is defined by the equations

$$0! = 1$$
$$s(y) = s(y) \times y!$$

To write this in our notation, we would for the first of these equations need a *nullary* function f that always returns 1. However, according to our definitions, we cannot define *any* nullary primitive recursive functions. So we will first define a *binary* function $\mathfrak{i} : \mathbb{N} \times \mathbb{N} \to \mathbb{N}$ such that $\mathfrak{i}(x, y) := y!$, and then simply discard the first "junk" argument. The binary "junk" factorial function is defined by the equations

$$\mathfrak{i}(x, 0) = 1$$
$$\mathfrak{i}(x, s(y)) = s(y) \times \mathfrak{i}(x, y)$$

We now want to find a unary function f and a ternary function g such that $\mathfrak{i} = \mathrm{Pr}[f, g]$. We know that $f(x) = 1$, so we may take $f(x) = s(z(x))$. Thus,

$$f = \mathrm{Cn}[s, z].$$

Next, we must find a ternary g such that

$$s(y) \times \mathfrak{i}(x, y) = g(x, y, \mathfrak{i}(x, y))$$

So g must be the product of the successor of the first argument and the third argument – i.e.,

$$g = \mathrm{Cn}[\times, \mathrm{Cn}[s, p^3_1], p^3_3].$$

So

$$\mathfrak{i} = \mathrm{Pr}[\mathrm{Cn}[s, z], \mathrm{Cn}[\times, \mathrm{Cn}[s, p^3_1], p^3_3]].$$

Finally, we can define the unary factorial function using, for example, $!(y) := \mathfrak{i}(y, y)$. This is easy:

$$! = \mathrm{Cn}[\mathfrak{i}, p^1_1, p^1_1].$$

Putting everything together, we get

$$! = \mathrm{Cn}[\mathrm{Pr}[\mathrm{Cn}[s, z], \mathrm{Cn}[\times, \mathrm{Cn}[s, p^3_1], p^3_3]], p^1_1, p^1_1].$$

6. *Truncated subtraction:* $x \dot{-} y$ defined by

$$x \dot{-} y = \begin{cases} 0 & \text{if } x < y; \\ x - y & \text{otherwise.} \end{cases}$$

(Exercise.)

7. *Distance function:* $|x - y|$ (Exercise.)

8. *Minimum and maximum functions:* $\min(x, y)$, $\max(x, y)$ (Exercise.)

9. *Bounded minimization:* If $f(x, \bar{z})$ is a primitive recursive function, then so is

$$(\min y < x) f(x, \bar{z}),$$

which returns the least $y < x$ such that $f(y, \bar{z}) = 0$ if such a y exists, and x otherwise.

Note that all primitive recursive functions are total. Moreover, it should be intuitively clear that all primitive recursive functions are *effectively* computable: given any n-ary function f on the natural numbers that is defined from the basic functions using composition and primitive recursion, and any input $x_1, \ldots, x_n$, I can compute the value $f(x_1, \ldots, x_n)$ in finitely many steps.

6.1.1 Primitive recursive sets and relations

Definition 6.5 (Primitive recursive sets and relations)**.** Let $S \subseteq \mathbb{N}$ be a set of natural numbers. Then S *is primitive recursive* if its characteristic function

$$\chi_S : \mathbb{N} \to \{0, 1\} \text{ s.t.}$$
$$\chi_S(n) = \begin{cases} 1 & \text{if } n \in S \\ 0 & \text{otherwise.} \end{cases}$$

is primitive recursive.

Similarly, a k-ary *relation* $R \subseteq \mathbb{N}^k$ is primitive recursive if its characteristic function $\chi_R : \mathbb{N}^k \to \{0, 1\}$ is primitive recursive.

Example 6.2. A very important part of the *arithmetization of syntax of first-order logic* that you saw during the first half of the course was to show that there are computable functions that determine whether a given natural number *codes* certain syntactic objects and properties. For example, the following relations are all primitive recursive:

1. $\mathrm{Fn}(x, n)$ iff x is the code of an n-ary function symbol;

2. $\mathrm{Term}(x)$ iff x is the Gödel number of a term;

3. $\mathrm{Sent}(x)$ iff x is the Gödel number of a sentence;

4. $\mathrm{Deriv}(d)$ iff d is the Gödel number of a correct derivation (proof) δ

5. $\mathrm{Prf}_\Gamma(x, y)$ (where Γ is a primitive recursive set of sentences) iff x codes a proof of A from Γ and y is the Gödel number of A;

6.2 The Ackermann function is not primitive recursive

We saw that a lot of familiar mathematical functions as well as meta-logical notions such as "x is a proof of y" are primitive recursive, and that primitive recursive functions are effectively computable. Could possibly *all* effectively computable functions be primitive recursive? The answer is no. Recall that in the last section we showed that addition, multiplication, and exponentiation are primitive recursive functions. These three are related to each other.

Addition is repeated addition of 1 – that is, repeated application of the successor operation:

$$a + n = a + \underbrace{s(s(\ldots(s(}_{n} 0\ ))\ldots)$$

Multiplication is repeated *addition*:

$$a \times n = \underbrace{a + a + \cdots + a}_{n}$$

Exponentiation is repeated *multiplication*:

$$\exp(a, n) = a^n = \underbrace{a \times a \times \cdots \times a}_{n}$$

We can continuing this process. For example, "super-exponentiation" (tetration) is repeated exponentiation:

$$a \Uparrow n = a^{a^{a^{\cdot^{\cdot^{\cdot}}}}} \text{ (a stack of } n \text{ } a\text{s)}$$

Here, the exponents are grouped with parentheses from the right. For example, $3 \Uparrow 3 = 3^{27}$. Super-duper-exponentiation (pentation?) is repeated super-exponentiation: $a \Uparrow^2 n = \underbrace{a \Uparrow (a \Uparrow (\cdots \Uparrow a))}_{n}$ In general, having defined $\Uparrow^n$, the next function $\Uparrow^{n+1}$: $\mathbb{N} \times \mathbb{N} \to \mathbb{N}$ is defined by the equation pair

$$x \Uparrow^{n+1} 0 = 1$$
$$x \Uparrow^{n+1} s(y) = x \Uparrow^n (x \Uparrow^{n+1} y)$$

Or, in [BBJ] notation,

$$\Uparrow^{n+1} = \mathrm{Pr}[\mathrm{Cn}[s, z], \mathrm{Cn}[\Uparrow^n, p_1^3, p_3^3].$$

This defines an infinite hierarchy of operations:

$$\begin{aligned} \ll 0 \gg &= + \\ \ll 1 \gg &= \times \\ \ll 2 \gg &= \exp \\ \ll 3 \gg &= \Uparrow^1 \\ \ll 4 \gg &= \Uparrow^2 \\ &\vdots \end{aligned}$$

Note that each $\ll n \gg$ is a primitive recursive function (we gave the definition above). Next, we define the ternary function α and the unary function γ on the natural numbers as follows:

$$\begin{aligned} \alpha(x, y, z) &:= x \ll y \gg z \\ \gamma(x) &:= \alpha(x, x, x) \end{aligned}$$

Note: the definition of α given above is not a definition by primitive recursion! (Why not?) The function γ thus defined is *not* the Ackermann function, but closely related to it. It grows very fast:

$$\begin{aligned} \gamma(0) &= 0 + 0 && = 0 \\ \gamma(1) &= 1 \times 1 && = 1 \\ \gamma(2) &= 2^2 && = 4 \\ \gamma(3) &= 3^{3^3} && = 7\ 625\ 597\ 484\ 987 \end{aligned}$$

It should be intuitively clear that α, and hence also γ, are effectively computable (we can compute its values!). However, it turns out that γ is *not* primitive recursive.

6.2.1 The Ackermann function is not primitive recursive (sketch)

I already claimed above that the definition I gave is not primitive recursive. But we will show (a sketch of the proof that) it is not possible at all to define γ primitive recursively. (Or, more precisely, that it is impossible to define α by primitive recursion.) And as we will see, the whole argument is yet another instance of diagonalization. Exciting!

First, we need to prove one thing:

Proposition 6.6. *It is impossible to obtain $+$ from the basic functions (zero, successor, projections) by composition, without using primitive recursion.*

Proof. Let f be an n-ary function that can be obtained from the basic function using only composition. We claim that there exists a positive integer a such that

$$f(\bar{x}) < (\max \bar{x}) + c \quad (\forall \bar{x} = x_1, \ldots, x_n).$$

No such c can exist for addition, because $(c+1)+(c+1) > (c+1)+c$. Therefore, addition cannot be obtained from the basic functions using only composition.

We prove the claim by induction: we show that it holds for the basic functions, and that it also holds for any function that is obtained from composing functions for which the claim holds.

Base case: The claim holds for the zero function (with $c = 1$), the successor function (with $c = 2$), and each projection function (with $c = 1$).

For the induction step, suppose f is a k-ary function and $g_1, \ldots, g_k$ are n-ary functions for which the claim holds. We want to show that the claim also holds for the function h obtained by composition of f with the g_i, i.e.

$$h(x_1, \ldots, x_n) = f(g_1(x_1, \ldots, x_n), \ldots, g_k(x_1, \ldots, x_n)).$$

For each g_i, by the induction hypothesis there is a_i such that

$$g_i(\bar{x}) < \max(\bar{x}) + a_i \quad (\forall \bar{x})$$

and similarly there is b such that

$$f(\bar{y}) < \max(\bar{y}) + b \quad (\forall \bar{y}).$$

Let $a := \max(\overline{a_i})$. Then for each g_i, we have

$$g_i(\bar{x}) < \max(\bar{x}) + a. \quad (\forall \bar{x})$$

So

$$\max(\overline{g_i(\bar{x})}) < \max(\bar{x}) + a \quad (\forall \bar{x}).$$

Therefore,

$$h(\bar{x}) = f(g_1(\bar{x}), \ldots, g_k(\bar{x})) < \max(\bar{x}) + a + b$$

so we may take $a + b$. □

Having done that, we now define for each n a unary function λ_n by setting $\lambda_n(x) := \alpha(x, n, x)$. Then we can use the infinite sequence of functions

$$\lambda_0, \lambda_1, \lambda_2, \ldots$$

to build a familiar table, which has the λ_n as rows and the natural numbers as columns:

x	0	1	2	3	$\cdots$
$\lambda_0(x)$	$0+0$	$1+1$	$2+2$	$3+3$	$\cdots$
$\lambda_1(x)$	0×0	1×1	2×2	3×3	$\cdots$
$\lambda_2(x)$	0^0	1^1	2^2	3^3	$\cdots$
$\lambda_3(x)$	0^{0^0}	1^{1^1}	2^{2^2}	3^{3^3}	$\cdots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$

Note that the diagonal (top-left to bottom-right) is exactly the sequence

$$\gamma(0), \gamma(1), \gamma(2), \gamma(3), \ldots$$

It should be clear that each λ_n grows faster for greater values of n. In fact, each primitive recursive function g is *dominated* by λ_n for some n (in the sense that λ_n eventually outgrows g), **depending on the number of times primitive recursion was used to define** g. For example, as we saw in the proof that addition cannot be defined without using primitive recursion at least once, the values of any primitive recursive function *not* using primitive recursion at all is upper bounded by addition with a constant, so any such function is in fact dominated by λ_0. So each application of primitive recursion bumps you up by (at most) one level, and composition doesn't bump the level at all. Since every primitive recursive function is built by finitely many applications of these two rules, every primitive recursive function sits below some finite level λ_n.

The function γ, however, is defined by iterating the previous operation of the hierarchy. And so $\gamma(n+1)$ uses primitive recursion exactly one more time than γ. So γ cannot be dominated by any one of the λ_n, because it "outruns" the λ_n along the diagonal. That's how diagonalization works: γ escapes the whole hierarchy because the hierarchy exhausts all primitive recursive functions, and γ eventually surpasses every individual level. We conclude that γ is not a primitive recursive function.

However, γ is effectively computable because we can compute its values (or write a Python script that does it). So, if we want to catch all effectively computable functions with recursion, we must add a new way of defining new recursive functions from old ones.

6.3 Recursive functions

Definition 6.7 (Unbounded minimization)**.** Given an $n+1$-ary function $f : \mathbb{N}^{n+1} \to \mathbb{N}$, the application of *unbounded minimization* (or *unbounded search* or the *μ-operator*), written $\mathrm{Mn}[f]$ or $\mu x\ f$, is the n-ary function $\mu x\ f : \mathbb{N}^n \to N$ such that

$$\mu x\ f(z_1, \ldots, z_n) := \begin{cases} x & \text{if } f(x, \bar{z}) = 0 \text{ and} \\ & \quad \forall t < x :\ f(t, \bar{z}) \text{ is defined and } f(t, \bar{z}) \neq 0; \\ \text{undefined} & \text{if no such } x \text{ exists.} \end{cases}$$

Note that the function $\mu x\ f$ may be partial even if f is total. So, while all *primitive* recursive functions are total, this is not the case for *recursive* functions.

Unbounded minimization is what we need to add to the primitive recursive functions to get (all) partial recursive functions. Since we are now dealing with partial functions, we need to slightly modify our definitions of composition and primitive recursion to account for partiality.

Definition 6.8 (Composition and primitive recursion for partial functions)**.** For partial functions f and g, write $f(\bar{x}) \simeq g(\bar{x})$ to mean that either $f(\bar{x})$ and $g(\bar{x})$ are both undefined, or they are both defined and equal.

Let $f : \mathbb{N}^k \rightharpoonup \mathbb{N}$ be a k-ary partial function and $g_1, \ldots, g_k : \mathbb{N}^n \rightharpoonup \mathbb{N}$ be n-ary partial functions. The *composition* of f with $g_1, \ldots, g_k$ is the n-ary partial function $h : \mathbb{N}^n \rightharpoonup \mathbb{N}$ such that

$$h(x_1, \ldots, x_n) \simeq f(g_1(x_1, \ldots, x_n), \ldots, g_k(x_1, \ldots, x_n)).$$

We set the convention that $h(\bar{x})$ is defined if and only if each g_i is defined at $\bar{x}$, and h is defined at $g_1(\bar{x}), \ldots, g_k(\bar{x})$.

We redefine *primitive recursion* similarly by just replacing $=$ with $\simeq$ in Definition 6.3.

Definition 6.9 (Partial recursive functions)**.** The set of *partial recursive functions* (or *μ-recursive functions* or just *recursive functions*) is the smallest set that contains the basic functions z, s, and p_i^n (for all $n \in \mathbb{N}$ and $1 \leq i \leq n$) and that is closed under composition, primitive recursion, and application of unbounded minimization.

The following few definitions amount to a terminological disaster, which is due to historical reasons.

Definition 6.10 (Total recursive functions)**.** A *recursive function* is a *total* partial recursive function.

Definition 6.11 (Regular function)**.** A total function $f(x, \bar{z})$ is *regular* if for all $\bar{z}$ there exists an x such that $f(x, \bar{z}) = 0$.

Definition 6.12 (General recursive functions)**.** The set of *general recursive functions* is the smallest set that contains the basic functions z, s, and p_i^n (for all $n \in \mathbb{N}$ and $1 \leq i \leq n$) and that is closed under composition, primitive recursion, and unbounded minimization applied to *regular* functions.

You encountered unbounded minimization applied to regular functions in the first half of the course.

It is clear that every general recursive function is a total recursive function. In fact, the converse also holds: Every total recursive function is a general recursive function. Thus, we get the following proposition:

Proposition 6.13 (General recursive and total recursive functions are the same)**.** *The general recursive functions and the total recursive functions are the same.*

So Definitions 6.10 and 6.12 really point to exactly the same functions.

6.4 Church's Thesis

Church's Thesis. *Any function that is effectively computable (computable by an algorithm) is partial recursive.*

6.5 [Excursion] The Ackermann function is recursive

In Section 6.2, we defined an infinite sequence of operations $\ll n \gg$ by primitive recursion and, based on that, we defined the functions

$$\alpha(x, y, z) := x \ll y \gg z;$$
$$\gamma(x) := \alpha(x, x, x).$$

We saw that α and γ are *not* primitive recursive. However, they are both effectively computable, and so if Church's Thesis is true they must both be partial recursive. But how? Here is a (very rough) sketch of a proof that α (and hence γ) can be defined as a μ-recursive function (as in Definition 6.9).

Remark. This proof follows essentially the same strategy as that which you used in the first half of the course to show that primitive recursive functions are in $\mathcal{C}$, using Gödel's Beta Function Lemma!

A *computation trace* for $\alpha(x, y, z) = v$ is a finite sequence of tuples

$$((x_1, y_1, z_1), v_1), ((x_2, y_2, z_2), v_2), \ldots, ((x_n, y_n, z_n), v_n)$$

such that:

- for each $i \leq n$, $\alpha(x_i, y_i, z_i) = v_i$;
- $x = x_n, y = y_n, z = z_n$, and $v = v_n$; and
- the sequence is ordered in the sense that each entry is justified by the recursive definition using only values that appear earlier in the sequence.

You know from the first half of the course that we can code any finite sequence using natural numbers, and check using primitive recursion whether a given number codes such a sequence. I claim that for a given number we can also check using primitive recursion whether that number codes a computation trace for $\alpha(x, y, z) = v$ – that is, whether it really justifies (or "proves") $\alpha(x, y, z) = v$. In fact, the strategy for this is *very* similar to the one you used during the first half of the course when you showed that the predicate

$$\mathrm{Prf}_\Gamma(x, y) :\iff x \text{ codes a proof of } A \text{ from } \Gamma \text{ and } y \text{ is the Gödel number of } A$$

is primitive recursive for any primitive recursive set of sentences Γ. Roughly, primitive recursive checking whether a sequence is a computation trace for $\alpha(x, y, z) = v$

works as follows: First, decode the sequence and its elements. Then check whether the last entry is $((x, y, z), v)$. This is easy.

Then, iterate through all entries of the sequence. Since there are only finitely many of them, this can be done primitive recursively. For each entry $((x_i, y_i, z_i), v_i)$, we need to check that it is justified. We do that as follows:

- If $y_i = 0$, check whether $v_i = x_i + z_i$. We have given a primitive recursive definition of addition, so we know this can be done primitive recursively.
- If $y_i = 1$, check whether $v_i = x_i \times z_i$. We have given a primitive recursive definition of multiplication, so we know this can be done primitive recursively.
- If $y_i > 1$ and $z_i = 0$, check whether $v_i = 1$.
- If $y_i > 1$ and $z_i > 1$: Then we need to "rewind" the recursive definition and check whether the sequence contains a justification. We know by the recursive definition of $\ll n \gg$ that

  $$\alpha(x_i, y_i, z_i) = \alpha(x_i, y_i - 1, \alpha(x_i, y_i, z_i - 1)).$$

  So we check (using bounded minimization, which is primitive recursive) whether there exists $j < i$ such that there is a previous entry

  $$((x_i, y_i, z_i - 1), w) \text{ for some } w$$

  in the sequence, and then whether there exists $k < i$ such that there is another previous entry

  $$((x_i, y_i - 1, w), v_i).$$

  If both are the case, we have found a justification. Otherwise, the sequence is not a computation trace for $\alpha(x, y, z) = v$.

I omitted many details, but I hope you are convinced that *verification* of a computation trace is primitive recursive.

Now it is *not* possible to *find* a computation trace for a given pair $((x, y, z), v)$ using primitive recursion only, *because we do not know how long such a trace might be*. But we can use unbounded search (the μ-operator) to do that! Namey, we define a ternary function find_trace by

$$\begin{aligned}\text{find\_trace}(x, y, z) := &\text{ the least number } t \text{ such that}\\ &t \text{ codes a computation trace for } \alpha(x, y, z) = v \text{ for some } v.\end{aligned}$$

If such a trace exists for (x, y, z), find_trace(x, y, z) will return it. Otherwise, it is undefined.

Finally, we can unpack the value $v = \alpha(x, y, z)$ using primitive recursion from the output of find_trace.

7 Turing computable and recursive functions are the same

In this chapter, we will prove that the Turing-computable functions (on the natural numbers) are exactly the same as the partial recursive functions (on the natural numbers). The fact that these two very different models of computation compute exactly the same functions gives some kind of evidence for the Church-Turing Thesis.

There are two directions that we need to prove:

1. Every partial recursive function is Turing-computable.
2. Every Turing-computable function is partial recursive.

7.1 Partial recursive functions are Turing-computable

We could go ahead and prove this direction directly from the definition. Recall Definition 6.9 of the partial recursive functions: they are obtained from the *basic functions*

- zero;
- successor;
- projections

and the following operations to obtain new partial recursive functions from old ones:

- composition;
- primitive recursion;
- minimization.

We would first have to show that all of the basic functions are Turing-computable. This is quite straightforward (in fact, you did that in the current homework assignment). Then, we would have to show that the Turing-computable functions are closed under composition, primitive recursion, and minimization – that is:

1. If $h = \mathrm{Cn}[f, g_1, \ldots, g_k]$ and $f, g_1, \ldots, g_k$ are all Turing-computable, then so is h.
2. If $h = \mathrm{Pr}[f, g]$ and f, g are Turing-computable, then so is h.
3. If $h = \mu x\ f(x, \bar{z})$ and f is Turing-computable, then so is h.

This is certainly doable, and in fact we will prove (1) and (3). However, while it is possible to prove (2), it involves a lot of bookkeeping. So we will choose another way to prove the equivalence, and thereby connect the material of this half of the course with a notion of computability that you have already encountered in the first half.

7.1.1 Refresher: $\mathcal{C}$

In the first half of the course, you also encountered a set of functions called $\mathcal{C}$. Here is the definition:

Definition 7.1 ($\mathcal{C}$)**.** The set $\mathcal{C}$ of functions is the smallest set that contains

- zero: $z(x) = 0$;
- successor: $s(x) = x + 1$;
- addition: $x + y$;
- multiplication: $x \times y$;
- projections: p_i^n;
- characteristic function for equality: $\chi_=(x, y)$

that is closed under

- composition and
- *regular* minimization (i.e., minimization applied to *regular* functions).

7.1.2 Connection between $\mathcal{C}$ and the total recursive functions

In the first half of the course, you proved (using Godel's β function) that:

Every primitive recursive function on the natural numbers is in $\mathcal{C}$.

The fact that every primitive recursive function is in $\mathcal{C}$ actually immediately yields a much stronger result:

Lemma 7.2. *Every general recursive function is in $\mathcal{C}$.*

Proof. By Definition 6.12, the *general recursive functions* are those obtained from the basic functions zero, successor and projections by applying composition, primitive recursion, and *regular* minimization.

The basic functions are all in $\mathcal{C}$ by definition. Likewise, $\mathcal{C}$ is closed under composition and regular minimization by definition. You *proved* that $\mathcal{C}$ is closed under primitive recursion. Therefore, every general recursive function is in $\mathcal{C}$. □

In fact we can also show the converse!

Lemma 7.3. *Every function in $\mathcal{C}$ is general recursive.*

Proof. First, note that all the "basic" functions of $\mathcal{C}$ are general recursive – in fact, even *primitive* recursive:

- zero: $z(x) = 0$;
- successor: $s(x) = x + 1$;
- projections: p_i^n

These are all primitive recursive by definition and hence general recursive. Moreover, we already showed previously (see Example 6.1) that the following are primitive recursive:

- addition: $x + y$;
- multiplication: $x \times y$;

Finally, the

- characteristic function for equality: $\chi_=(x, y)$

is primitive recursive because it can be defined as

$$\chi_=(x, y) := \begin{cases} 1 & \text{if } x \dot{-} y = 0 \text{ and } y \dot{-} x = 0; \\ 0 & \text{otherwise.} \end{cases}$$

and we know that truncated subtraction $\dot{-}$ is primitive recursive (you showed that in a homework problem).

Any function obtained by applying composition or regular minimization to primitive recursive functions is general recursive by Definition 6.12 of *general recursive function*. □

By Proposition 6.13, the *general* recursive functions and the *total* recursive functions are the same. Summing up, we have:

Theorem 7.4. *The set $\mathcal{C}$ is precisely the set of general/total recursive functions.*

Proof. By Lemmas 7.2 and 7.3. □

7.1.3 Partial recursive functions are Turing-computable

We can now use the fact that the set of total recursive functions is precisely the set $\mathcal{C}$ to give a much simpler proof that the *total* recursive functions are all Turing-computable. After that, it will not be hard to show that this generalizes in the sense that all *partial* recursive functions are Turing-computable.

Lemma 7.5. *The "basic" functions of $\mathcal{C}$ are all Turing-computable.*

Proof.

- **Zero function** z: on an empty tape, write one stroke and halt

- **Successor function** s: given a block of n strokes, move to the right until you scan the first B, print 1, then move left to first stroke and halt
- **Projections**: Construct a TM T that computes p_i^n:
  - move right to beginning of ith block, erasing first $i-1$ blocks on the way
  - move right until first blank
  - move right, erasing $n-i$ blocks
  - move left until scan stroke
  - move left until scan blank
  - move right
- **Addition**: Given two blocks of strokes,
  - fill in the separator B with a 1
  - move to beginning of block
  - erase two strokes
  - and halt on first stroke
- **Multiplication**: Given two blocks of strokes, the idea is to use left block as a counter.
  - Create a copy of the right block (separated to the left by one B)
  - Loop: While leftmost block contains more than two strokes,
    - Copy second block from the left to the right of the last block (no separator)
    - Erase leftmost stroke of leftmost block
    - Go back to beginning of loop
  - If leftmost block contains two or less strokes:
    - Erase that stroke
    - Erase block to the right
    - Move to next stroke to the right and halt
- **Characteristic function for equality**: given two blocks of strokes separated by a single blank,
  - iteratively remove first stroke if any left
  - if both blocks run out at the same time, print two strokes, move left, halt
  - if one blocks runs out before the other, erase the remaining block, print one stroke, halt

□

Lemma 7.6. *The set of Turing-computable partial functions is closed under composition.*

Proof. Homework. □

Lemma 7.7. *The set of Turing-computable functions is closed under minimization.*

That is, if the function $f(x, \bar{z})$ is Turing-computable, then so is the function $\mu x\ f(x, \bar{z})$.

Proof. Suppose we have a machine T_f that computes $f(x, \bar{z})$. We construct a machine T that computes $\mu x\ f(x, \bar{z})$ as follows: The idea is to iteratively compute $f(0, \bar{z}), f(1, \bar{z}), \ldots$ until the output is 0. We do this as follows: Let $\mathcal{I}$ denote the tape segment encoding $\bar{z}$. Loop as follows (step x):

- Assume the tape content is of the form $\mathcal{I}B1^{x+1}$. Append a B, copy 1^{x+1}, append another B, and copy $\mathcal{I}$:
$$\mathcal{I}B1^{x+1}B1^{x+1}B\mathcal{I}$$
- Run T_f on the last two blocks (i.e. starting at the first stroke of the second block of 1^{x+1}):
$$\mathcal{I}B1^{x+1}B1^{f(x,\bar{z})+1}$$
- If $f(x, \bar{z}) = 0$ (i.e., if the last block on the tape consists of exactly one stroke), erase everything and then halt with output x (i.e., $x + 1$ strokes).
- Otherwise, delete the block $1^{f(x,\bar{z})+1}$ and add a stroke to the block 1^{x+1}:
$$\mathcal{I}B1^{x+2}$$
- Go to the beginning of the loop.

□

Summing up:

Lemma 7.8. *Every function in $\mathcal{C}$ is Turing-computable.*

Proof. Immediate by Lemmas 7.5, 7.6 and 7.7. □

And finally:

Proposition 7.9. *Every total recursive function is Turing-computable.*

Proof. By Theorem 7.4, the set of total recursive functions is precisely the set $\mathcal{C}$. By Lemma 7.8, the functions in $\mathcal{C}$ are Turing-computable. So every total recursive function is Turing-computable. □

This deals only with the *total* recursive functions. But we can generalize:

Corollary 7.10. *Every partial recursive function is Turing-computable.*

Proof. Homework. □

7.1.4 Refresher: Representability in Q

This subsection and the next connects two big themes from the first and the second half of the course: We will show that *representability in Robinson's Arithmetic* **Q** is the same as *total recursiveness*. Thus, representability in **Q** adds another model of computation to our list (next to Turing machines and recursive functions)!

Recall the theory **Q** of Robinson's Arithmetic and the notion of *representability in* **Q**. I will give the definition of representability below. Recall also the following notation convention: For a number $n \in \mathbb{N}$, write

$$\overline{n} \quad \text{for} \quad \underbrace{s(\dots s}_{n}(0)).$$

Definition 7.11 (Representability in **Q**)**.** A function $f(x_1, \dots, x_k) : \mathbb{N} \to \mathbb{N}$ is *representable in* **Q** if there is a formula $\varphi_f(x_1, \dots, x_k, y)$ such that, whenever $f(n_1, \dots, n_k) = m$, we have

1. $\mathbf{Q} \vdash \varphi_f(\overline{n_0}, \dots, \overline{n_k}, \overline{m})$ and
2. $\mathbf{Q} \vdash \forall y\ (\varphi_f(\overline{n_1}, \dots, \overline{n_k}, y) \to \overline{m} = y)$.

7.1.5 Connection between total recursive functions and functions representable in Q

In the first half of the course, you showed that

Lemma 7.12. *Every function in $\mathcal{C}$ is representable in* **Q**.

Proof. You proved this in the first half of the course. □

We will now prove the converse: Every function representable in **Q** is total recursive. First, we need a lemma.

Lemma 7.13. *If $f(x_1, \dots, x_k)$ is representable in* **Q**, *then there is a formula $\varphi_f(x_1, \dots, x_k, y)$ such that*

$$\mathbf{Q} \vdash \varphi_f(\overline{n_1}, \dots, \overline{n_k}, \overline{m}) \iff m = f(n_1, \dots, n_k).$$

Proof. (Sketch.)

$\Longleftarrow$: Immediate by Definition 7.11 part (1).

$\Longrightarrow$: Follows from Definition 7.11 part (2) and consistency of $\mathbf{Q}$. (For more detail, see proof of Lemma 4.3 in [IC].) □

Lemma 7.14 (Functions representable in $\mathbf{Q}$ are in $\mathcal{C}$)**.** *Every function that is representable in* $\mathbf{Q}$ *is total recursive.*

Proof. (Sketch. For more detail, see Section 4.2 in [IC].)

First, recall from the first half of the course that we can encode and decode finite sequences of natural numbers primitive recursively. If s is the Gödel number of a sequence $\langle s_1, \ldots, s_n \rangle$, write $(s)_i$ for the (decoding of the) ith element s_i.

Let $f : \mathbb{N}^k \to \mathbb{N}$ be a function represented by $\varphi_f(x_1, \ldots, x_k, y)$.

Idea: On input $n_1, \ldots, n_k$:

- Iterate through natural numbers
- For each n:
  - Check if n codes a sequence $\langle s_0, s_1 \rangle$ of natural numbers of length two such that $\mathrm{Prf}_{\mathbf{Q}}(s_0, s_1)$ – i.e. s_0 is the Gödel number of a correct derivation of the Gödel number of the formula $\varphi_f(\overline{n_1}, \ldots, \overline{n_k}, \overline{(s)_1})$. (This can be done primitive recursively.)
  - If yes, then $(s)_1 = f(n_1, \ldots, n_k)$ by Lemma 7.13.

So

$$f(n_1, \ldots, n_k) = \left(\mu s \, \mathrm{Prf}_{\mathbf{Q}}\left((s)_0, \varphi_f(\overline{n_1}, \ldots, \overline{n_k}, (s)_1)\right)\right)_1 .$$

□

Summing up, we established the following equality:

$$\begin{aligned} \text{Functions representable in } Q &= \mathcal{C} \\ &= \text{total recursive functions} \\ &= \text{general recursive functions.} \end{aligned}$$

7.2 Turing-computable functions are partial recursive

In this section, we show the other direction of computational equality: every Turing-computable partial function is partial recursive. We will show that all notions related to *computations of Turing machines* (in the sense of Definition 3.7) are recursive – and, in fact, even primitive recursive, and that *existence* of a suitable computation sequence can be defined by one application of minimization.

Since we only deal with recursive functions *on natural numbers*, we first need to represent all relevant TM concepts and notions numerically.

7.2.1 Numerically representing TMs and TM configurations

Recall Definition 3.1 of a Turing machine:

Definition. A *Turing machine (TM)* is a quadruple $T = (Q, \Gamma, \delta, q_0)$, where

- Q is a finite set of *states*;
- Γ is a finite set of *symbols* including a symbol B called "blank";
- $\delta : Q \times \Gamma \rightharpoonup Q \times \Gamma \cup \{L, R\}$ is a partial function called the *transitioning function*; and
- $q_0 \in Q$ is a state called the *initial state*.

Given a TM $T = (Q, \Gamma, \delta, q_0)$, we can represent it in terms of (finite sequences of) natural numbers as follows: Suppose T has $|Q| = n$ states and $|\Gamma| = m$ symbols. We represent its

- *states* as numbers $0, \ldots, n-1$, where q_0 is 0;
- *symbols* as numbers $0, \ldots, m-1$, where B is 0;
- Represent L ("move left" instruction) as number m;
- Represent R ("move right" instruction) as number $m+1$;

Then we can represent the transitioning function δ as a partial function from

$$\{0, \ldots, n-1\} \times \{0, \ldots, m-1\}$$

to

$$\{0, \ldots, n-1\} \times \{0, \ldots, m+1\}.$$

Now recall Definition 3.2 of a configuration:

Definition. A *configuration* of a Turing machine $T = (Q, \Gamma, \delta, q_0)$ is a quadruple $(\alpha, q, \gamma, \beta)$, where

- α is the string of symbols to the *left* of the reading head of T starting from the first non-blank symbol of the tape if there *are* non-blank symbols to the left of the reading head or B otherwise;
- β is the string of symbols to the *right* of the reading head of T ending at the last non-blank symbol of the tape if there *are* non-blank symbols to to the right of the reading head or B otherwise;
- $\gamma \in \Gamma$ is the symbol *under* the reading head of T; and
- $q \in Q$ is the current *state* of T.

In our numerical representation, a configuration $(\alpha, q, \gamma, \beta)$ then consists of

- α, β: strings of natural numbers;
- q: natural number;
- β: string of natural numbers.

7.2.2 Encoding computations of TMs

Here is the general idea: We will show that

- the notion of being a halting computation is a primitive recursive relation;
- the function that returns the output of a halting computation is primitive recursive;

Then if f is computed by a TM T, we can describe f as a partial recursive function as follows:

- on input $\bar{x}$, use minimization to look for a halting computation sequence for machine T on input x;
- if there is one, return the output of the computation sequence.

I will take for granted the following primitive recursive *relations*[5]:

- Orderings $<, \leq$ of natural numbers;
- Boolean operations like $\wedge, \vee$ to combine primitive recursive relations

I will also take for granted a number of primitive recursive operations on finite sequences[6]. In particular:

- Finite sequences can be coded by natural numbers; write $\langle s_1, \ldots, s_n \rangle$ for the number coding the sequence $(s_1, \ldots, s_n)$
- Primitive recursive operations on finite sequences:
  - $length(n)$: if n codes a finite sequence, return its length, otherwise 0
  - $(n)_i$: if n codes a finite sequence of length at least i, return the ith element

In the following, all names of *functions* will start with *lowercase letters* and all names of *relations* will start with *uppercase letters*.

[5] If you want to see how these can be defined primitive recursively, see [IC] Section 2.8 or [BBJ] Section 7.1.

[6] See [IC] Section 2.11.

For instructions We will code instructions of δ as quadruples (q, γ, q', x), meaning $\delta(q, \gamma) = (q', x)$.

$$\begin{aligned} \mathit{Instruction}(k, n, m) :\Longleftrightarrow\ & \mathit{length}(k) = 4 \ \wedge\ (k)_1 < n \ \wedge\ (k)_2 < m \\ & \wedge\ (k)_3 < n \ \wedge\ (k)_4 < m + 2 \end{aligned}$$

This relation holds iff the number k codes a suitable instruction for a Turing machine with n states and m symbols.

$$\begin{aligned} \mathit{iState}(k) &:= (k)_1 \\ \mathit{iSymbol}(k) &:= (k)_2 \\ \mathit{iNextState}(k) &:= (k)_3 \\ \mathit{iAction}(k) &:= (k)_4 \end{aligned}$$

These four functions return the corresponding components of an instruction.

$$\begin{aligned} &\mathit{InstructionSeq}(s, n, m) :\Longleftrightarrow \\ &\quad \forall i \le \mathit{length}(s)\big(\\ &\qquad \mathit{Instruction}((s)_i, n, m) \wedge \\ &\qquad \forall i \le \mathit{length}(s)\ \forall j \le \mathit{length}(s)\big(\\ &\qquad\quad \big(\mathit{iState}((s)_i) = \mathit{iState}((s)_j) \ \wedge\ \mathit{iSymbol}((s)_i) = \mathit{iSymbol}((s)_j)\big) \to i = j \\ &\qquad\big) \\ &\quad\big) \end{aligned}$$

This relation is holds iff s codes a suitable sequence of instructions for a TM with n states and m symbols. The main requirement is that the list of quadruples corresponds to a function: for any symbol and state, there is at most one instruction that applies.

For Turing machines We represent a Turing machine T as a triple (n, m, σ), where n, m are natural numbers and σ is a sequence of quadruples of natural numbers.

$$TM(T) :\Longleftrightarrow \mathit{length}(T) = 3 \ \wedge\ \mathit{InstructionSeq}((T)_3, (T)_1, (T)_2)$$

This relation holds iff the number T codes a TM.

$$\begin{aligned} \mathit{tNumStates}(T) &:= (T)_1 \\ \mathit{tNumSymbols}(T) &:= (T)_2 \\ \mathit{tInstructions}(T) &:= (T)_3 \end{aligned}$$

These return the corresponding components of a TM.

$$tLeft(T) := tNumSymbols(T)$$
$$tRight(T) := tNumSymbols(T) + 1$$

These return the representatives for the "move left" and "move right" actions.

For strings (sequences) of symbols

$$firstSymbol(s) := \begin{cases} (s)_1 & \text{if } length(s) > 0; \\ 0 & \text{otherwise.} \end{cases}$$
$$lastSymbol(s) := \begin{cases} (s)_{length(s)} & \text{if } length(s) > 0; \\ 0 & \text{otherwise.} \end{cases}$$

$$addSymbolAtBeginning(a, s) := ...$$
$$addSymbolAtEnd(a, s) := ...$$

These return the number codes of the sequences $(a, (s)_1, \ldots, (s)_{length(s)})$ and $((s)_1, \ldots, (s)_{length(s)}, a)$, respectively.[7]

$$dropSymbolAtBeginning(s) := ...$$
$$dropSymbolAtEnd(s) := ...$$

These return the number codes of dropping the first (last) symbol of the sequence coded by s if s is non-empty, and s otherwise.

For configurations

$$\begin{aligned} Configuration(T, c) :\Longleftrightarrow\ & length(c) = 4 \\ & \wedge\ \forall i \leq length((c)_1)_i < tNumSymbols(T) \\ & \wedge\ (c)_2 < tNumStates(T) \\ & \wedge\ (c)_3 < tNumSymbols(T) \\ & \wedge\ \forall i \leq length((c)_4)_i < tNumSymbols(T) \end{aligned}$$

This relation holds iff T codes a TM and c codes a configuration of T.

[7] We haven't introduced the required primitive recursive machinery to deal with finite sequences to give the precise definitions of these primitive recursive functions.

$$
\begin{aligned}
cLeftString(c) &:= (c)_1 \\
cState(c) &:= (c)_2 \\
cSymbol(c) &:= (c)_3 \\
cRightString(c) &:= (c)_4
\end{aligned}
$$

These return the corresponding components of a configuration.

$$changeState(c, q) := \langle cLeftString(c), cState(c), cSymbol(c), cRightString(c) \rangle$$

This returns the (code of the) configuration obtained by changing its state to q.

$$
\begin{aligned}
moveLeft(c) := \langle & \\
& dropSymbolAtEnd(cLeftString(c)), \\
& cState(c), \\
& lastSymbol(cLeftString(c)), \\
& addSymbolAtBeginning(cSymbol(c), cRightString(c)) \\
\rangle & \\
moveRight(c) := & \ldots
\end{aligned}
$$

These return the result of moving left (resp. right) in the configuration coded by c.

$$
\begin{aligned}
HaltingConfig(T, c) :\Longleftrightarrow \neg\exists i \leq & \, length(tInstructions(T)) \\
& \big(iState((tInstructions(T))_i) = cState(c) \\
& \quad \wedge \; iSymbol((tInstructions(T))_i) = cSymbol(c)\big)
\end{aligned}
$$

This holds iff c is the code of a halting configuration for the TM T coded by T.

$$
\begin{aligned}
nextInstNum(T, c) := & \min i < length(tInstructions(T)) \\
& \big(iState((tInstructions(T))_i) = cState(c) \\
& \quad \wedge \; iSymbol((tInstructions(T))_i) = cSymbol(c)\big) \\
nextInst(T, c) := & \, (tInstructions(T))_{nextInstNum(T,c)}
\end{aligned}
$$

Assuming that c is not a halting configuration, these return the index of the next instruction for T in configuration c, and the instruction itself.

$$nextConfig(T, c) := \ldots$$

Assuming that c is not a halting configuration, this returns the next configuration for T after c.

$$initialConfig(x) := \ldots$$

This returns the initial configuration for the input coded by x.

For computation sequences

$$\begin{aligned} CompSeq(s,T,x) :\Longleftrightarrow\ & (s)_1 = initialConfig(x) \\ & \wedge\ \forall i \leq length(s) - 1\big(\\ & \quad \neg HaltingConfig(T,(s)_i)\ \wedge\ (s)_{i+1} = nextConfig(T,(s)_i) \\ & \big)\ \wedge\ HaltingConfig(T,(s)_{length(s)}) \end{aligned}$$

This holds iff s codes a halting computation sequence of the TM coded by T on the input coded by x.

$$cOutput(c) := \ldots$$

This returns the output represented by configuration c, according to out output encoding conventions.

$$output(s) := cOutput((s)_{length(s)})$$

This returns the output of the last configuration of the computation sequence coded by s.

7.2.3 Theorem

Theorem 7.15. *Every Turing-computable function is partial recursive.*

Proof. Let f be a partial function, and T a TM that computes f. Then for every $\bar{x}$, we have

$$f(\bar{x}) = output(\mu s\ CompSeq(s,T,\langle\bar{x}\rangle)).$$

□

The proof that every Turing-computable function is partial recursive goes a long way towards explaining why we believe that *every* (effectively) computable function is partial recursive. Intuitively, something is computable if one can describe a method of computing it, breaking the algorithm down into small and easily verifiable steps. The preceding proof shows that any algorithm for which it is primitive recursively verifiable that something is a state and that the transition from one state to another is correct is partial recursive.

7.2.4 Equivalence

Putting everything we did in this section together, we get:

Theorem 7.16. *The set of partial recursive functions on the natural numbers is precisely the set of Turing-computable functions on the natural numbers.*

Proof. Immediate from Corollary 7.10 and Theorem 7.15. □

Since Turing-computability and recursiveness are the same, we can just use the term "computable" to mean either.

Definition 7.17 (Computable functions and relations)**.** A function f on the natural numbers is *computable* iff it is Turing-computable or, equivalently, partial recursive. A relation R on the natural numbers is *computable* iff its characteristic function is computable.

8 Universal computation

When we proved Theorem 7.15 (every Turing-computable function is partial recursive) by showing that the notion of a *computation sequence* of a Turing machine is primitive recursive and that a suitable computation sequence for a given function argument can be searched for using minimization, we in fact proved yet another, very strong theorem:

Theorem 8.1 (Kleene's Normal Form Theorem)**.** *There are a primitive recursive relation $T(s, M, x)$ and a primitive recursive function $U(s)$ with the following property: if f is any partial computable function, then there exists an M such that*

$$\textit{there exists an } M \textit{ s.t. } f(x) \simeq U(\mu s\ T(s, M, x)) \textit{ for every } x.$$

Proof. T and U are simply historical notations for any relation and function pair that behaves like our *CompSeq* and *ouput* from Section 7.2.2. □

Corollary 8.2 (Universal Turing Machine)**.** *There exists a Turing machine V such that, for every pair of natural numbers (m, n): If m codes a Turing machine T, then V halts on input (m, n) if and only if T halts on input n. Moreover, if T halts on input n with output y, then V halts on input (m, n) with output y.*

Proof. Suppose m codes Turing machine T. Define the binary function

$$g(m, n) := U(\mu s\ T(s, m, n)),$$

where U and T are as in Theorem 8.1. Then by the same Theorem, g is a partial recursive function and $g(m, n)$ is defined if and only if T halts on input n and $g(m, n)$

is the output of T on n. By Theorem 7.16 (partial recursive functions and Turing-computable functions are the same), g is Turing-computable. This means that there is a Turing machine V that computes g. □

Corollary 8.2 says that there is a *universal Turing machine* which takes the *code* of an arbitrary Turing machine and and input and returns the output *of that Turing machine on that input*. The existence of such a Turing machine was proved in Turing's 1936 paper in which he first introduced Turing machines, and it is the theoretical proof of the possibility of something which was built only about a decade later: **the modern computer**, which can execude code on input, while both of them are stored on the same storage device and can be modified.

In simple terms, Corollary 8.2 says that *there exist computing devices on which you can "install" arbitrary software.*

9 TODO Undecidability of the Entscheidungsproblem

10 TODO Church-Turing Thesis

11 Potential further topics

11.1 The smn theorem

11.2 Rice's theorem

11.3 Representability in arithmetic as a model of computation

11.4 Fixed point theorem

11.5 Kleene's recursion theorem

11.6 Incompleteness via halting problem